

4x4 and the number of on pixels are counted in each block. This generates an input matrix of 8x8 where each element is an integer in the range 0..16. This reduces dimensionality and gives invariance to small distortions.

For info on NIST preprocessing routines, see M. D. Garriss, J. L. Blue, G. T. Candela, D. L. Dimmick, J. Geist, P. J. Grother, S. A. Janet, and C. L. Wilson, NIST Form-Based Handprint Recognition System, NISTIR 5469, 1994.

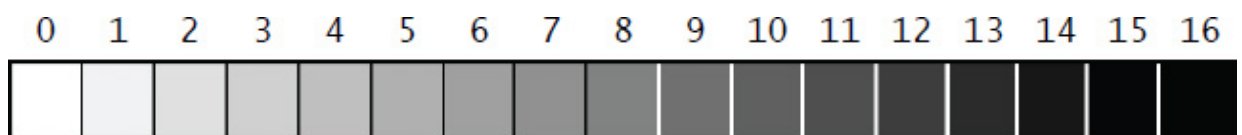
.. topic:: References

- C. Kaynak (1995) Methods of Combining Multiple Classifiers and Their Applications to Handwritten Digit Recognition, MSc Thesis, Institute of Graduate Studies in Science and Engineering, Bogazici University.
- E. Alpaydin, C. Kaynak (1998) Cascading Classifiers, Kybernetika.
- Ken Tang and Ponnuthurai N. Suganthan and Xi Yao and A. Kai Qin. Linear dimensionality reduction using relevance weighted LDA. School of Electrical and Electronic Engineering Nanyang Technological University. 2005.
- Claudio Gentile. A New Approximate Maximal Margin Classification Algorithm. NIPS. 2000.

hecking the Sample and Target Sizes

The Bunch object's **data** and **target attributes** are NumPy arrays:

- The data array contains the 1797 samples (the digit images), each with 64 features, having values in the range 0–16, representing *pixel intensities*. With Matplotlib, we'll visualize these intensities in grayscale shades from white (0) to black (16):



- The target array contains the images' labels—that is, the classes indicating which digit each image represents. The array is called target because, when you make predictions, you're aiming to “hit the target” values. To see labels of samples throughout the dataset, let's display the target values of every 100th sample:

[lick here to view code image](#)

```
In [4]: digits.target[::100]
Out[4]: array([0, 4, 1, 7, 4, 8, 2, 2, 4, 4, 1, 9, 7, 3, 2, 1, 2, 5])
```

We can confirm the number of samples and features (per sample) by looking at the data array's `shape` attribute, which shows that there are 1797 rows (samples) and 64 columns (features):

[lick here to view code image](#)

```
In [5]: digits.data.shape
Out[5]: (1797, 64)
```

You can confirm that the number of target values matches the number of samples by looking at the `target` array's `shape`:

[lick here to view code image](#)

```
In [6]: digits.target.shape
Out[6]: (1797,)
```

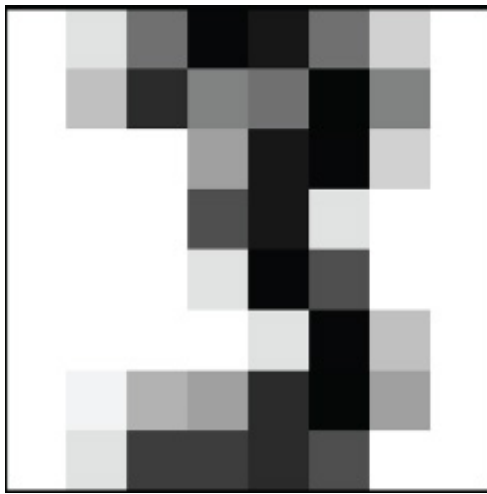
A Sample Digit Image

Each image is two-dimensional—it has a width and a height in pixels. The `Bunch` object returned by `load_digits` contains an `images` attribute—an array in which each element is a two-dimensional 8-by-8 array representing a digit image's pixel intensities. Though the original dataset represents each pixel as an integer value from 0–16, scikit-learn stores these values as *floating-point* values (NumPy type `float64`). For example, here's the two-dimensional array representing the sample image at index 13:

[lick here to view code image](#)

```
In [7]: digits.images[13]
Out[7]:
array([[ 0.,  2.,  9., 15., 14.,  9.,  3.,  0.],
       [ 0.,  4., 13.,  8.,  9., 16.,  8.,  0.],
       [ 0.,  0.,  0.,  6., 14., 15.,  3.,  0.],
       [ 0.,  0.,  0., 11., 14.,  2.,  0.,  0.],
       [ 0.,  0.,  0.,  2., 15., 11.,  0.,  0.],
       [ 0.,  0.,  0.,  0.,  2., 15.,  4.,  0.],
       [ 0.,  1.,  5.,  6., 13., 16.,  6.,  0.],
       [ 0.,  2., 12., 12., 13., 11.,  0.,  0.]])
```

and here's the image represented by this two-dimensional array—we'll soon show the code for displaying this image:



Preparing the Data for Use with Scikit-Learn

Scikit-learn's machine-learning algorithms require samples to be stored in a *two-dimensional array of floating-point values* (or two-dimensional *array-like* collection, such as a list of lists or a pandas `DataFrame`):

- Each row represents one *sample*.
- Each column in a given row represents one *feature* for that sample.

To represent every sample as one row, multi-dimensional data like the two-dimensional image array shown in snippet [7] must be *flattened* into a one-dimensional array.

If you were working with a data containing **categorical features** (typically represented as strings, such as 'spam' or 'not-spam'), you'd also have to *preprocess* those features into numerical values—known as one-hot encoding, which we cover in the next chapter. Scikit-learn's **`sklearn.preprocessing`** module provides capabilities for converting categorical data to numeric data. The Digits dataset has no categorical features.

For your convenience, the `load_digits` function returns the preprocessed data ready for machine learning. The Digits dataset is numerical, so `load_digits` simply flattens each image's two-dimensional array into a one-dimensional array. For example, the 8-by-8 array `digits.images[13]` shown in snippet [7] corresponds to the 1-by-64 array `digits.data[13]` shown below:

[lick here to view code image](#)

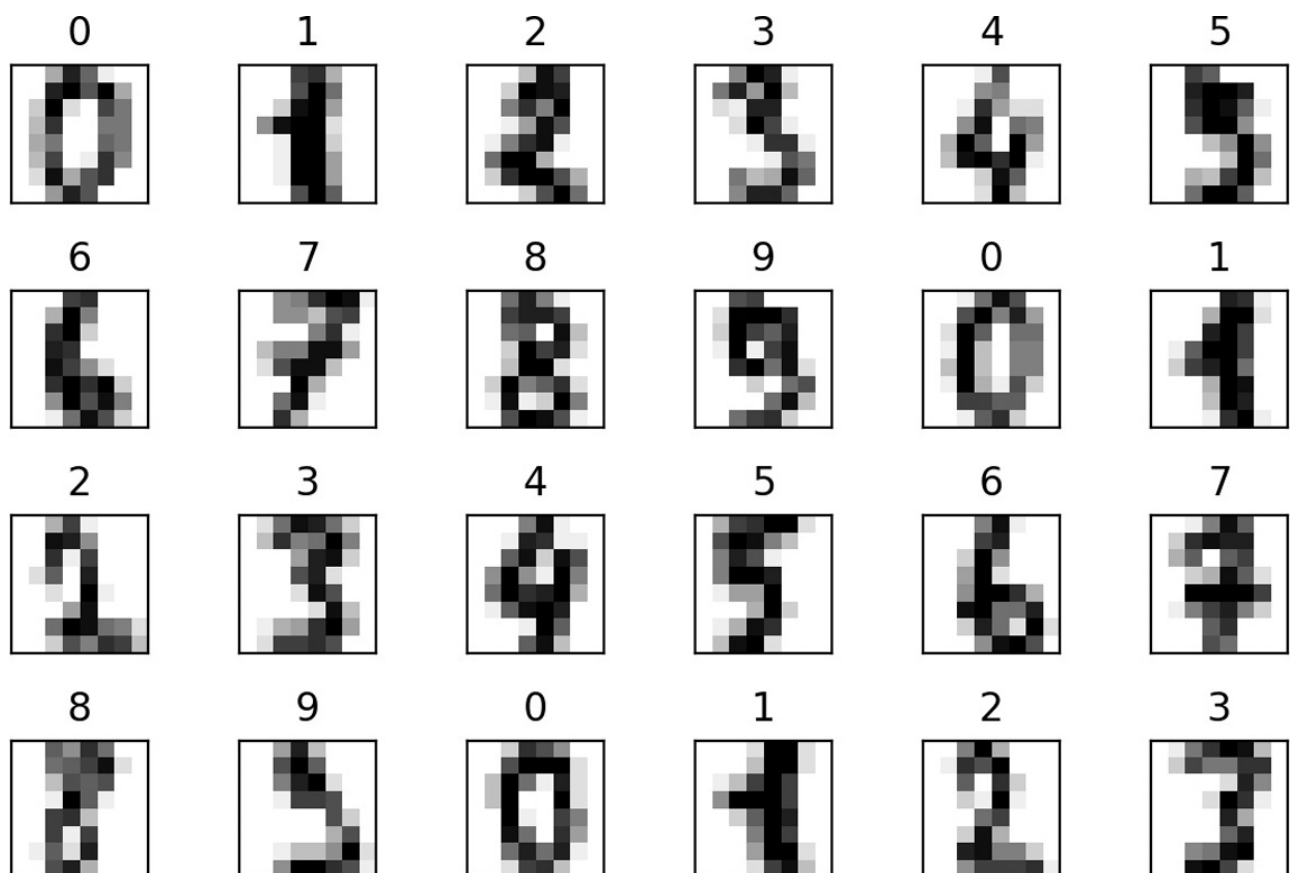
```
In [8]: digits.data[13]
Out[8]:
array([ 0.,  2.,  9., 15., 14.,  9.,  3.,  0.,  0.,  4., 13.,  8.,  9.,
```

```
16., 8., 0., 0., 0., 0., 6., 14., 15., 3., 0., 0., 0.,
0., 11., 14., 2., 0., 0., 0., 0., 0., 2., 15., 11., 0.,
0., 0., 0., 0., 0., 2., 15., 4., 0., 0., 1., 5., 6.,
13., 16., 6., 0., 0., 2., 12., 12., 13., 11., 0., 0.] )
```

In this one-dimensional array, the first eight elements are the two-dimensional array's row 0, the next eight elements are the two-dimensional array's row 1, and so on.

14.2.3 Visualizing the Data

You should always familiarize yourself with your data. This process is called **data exploration**. For the digit images, you can get a sense of what they look like by displaying them with the Matplotlib `imshow` function. The following image shows the dataset's first 24 images. To see how difficult a problem handwritten digit recognition is, consider the *variations* among the images of the 3s in the first, third and fourth rows, and look at the images of the 2s in the first, third and fourth rows.



Creating the Diagram

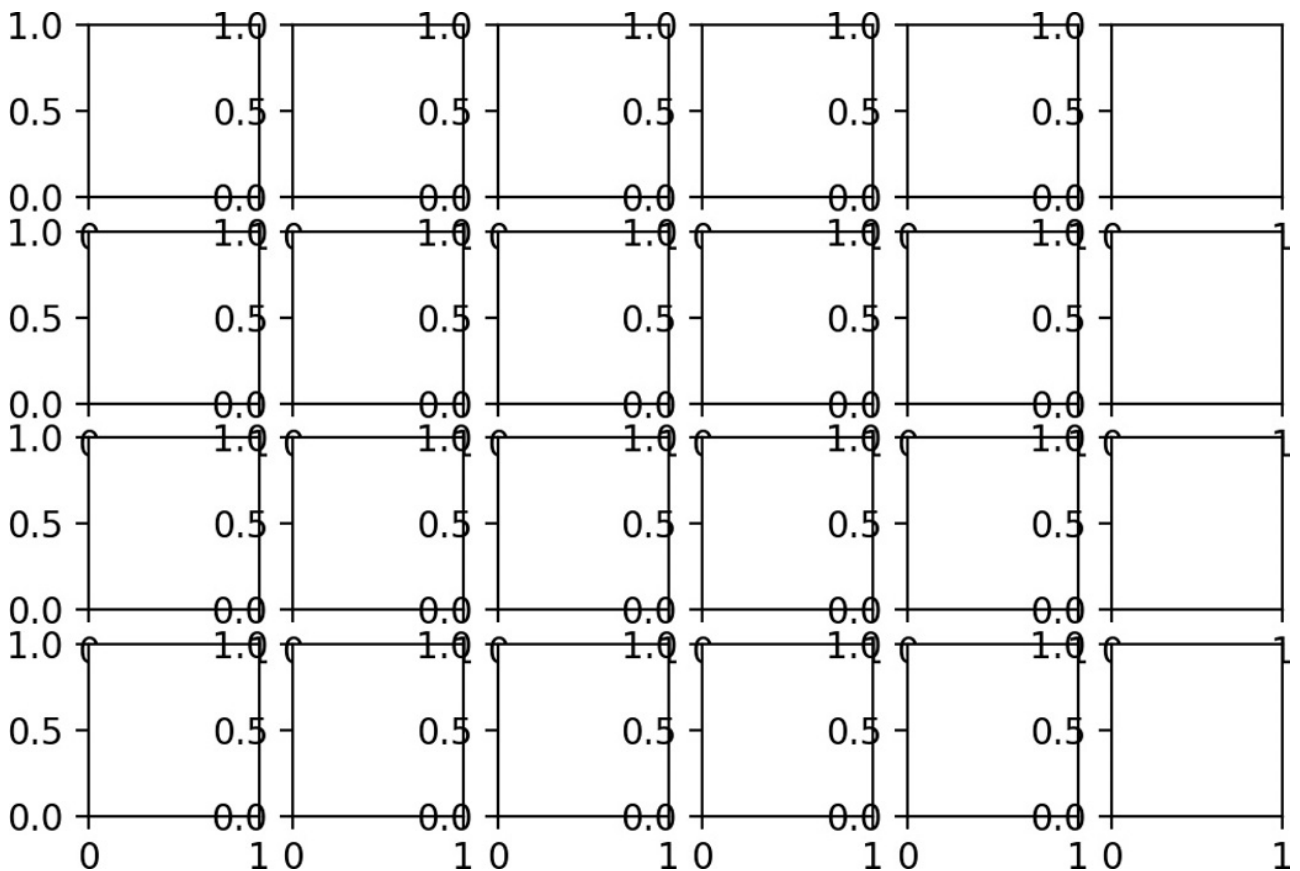
Let's look at the code that displayed these 24 digits. The following call to function `subplots` creates a 6-by-4 inch Figure (specified by the `figsize(6, 4)` keyword argument) containing 24 subplots arranged in 4 rows (`nrows=4`) and 6 columns (`ncols=6`). Each subplot has its own `Axes` object, which we'll use to display one digit image:

[lick here to view code image](#)

```
In [9]: import matplotlib.pyplot as plt

In [10]: figure, axes = plt.subplots(nrows=4, ncols=6, figsize=(6, 4))
```

Function `subplots` returns the `Axes` objects in a two-dimensional NumPy array. Initially, the `Figure` appears as shown below with labels (which we'll remove) on every subplot's x - and y -axes:



Displaying Each Image and Removing the Axes Labels

Next, use a `for` statement with the built-in `zip` function to iterate in parallel through the 24 `Axes` objects, the first 24 images in `digits.images` and the first 24 values in `digits.target`:

[lick here to view code image](#)

```
In [11]: for item in zip(axes.ravel(), digits.images, digits.target):
...:     axes, image, target = item
...:     axes.imshow(image, cmap=plt.cm.gray_r)
...:     axes.set_xticks([]) # remove x-axis tick marks
...:     axes.set_yticks([]) # remove y-axis tick marks
...:     axes.set_title(target)
...: plt.tight_layout()
```

```
...:
...:
```

Recall that NumPy array method `ravel` creates a *one-dimensional view* of a multidimensional array. Also, recall that `zip` produces tuples containing elements from the same index in each of `zip`'s arguments and that the argument with the fewest elements determines how many tuples `zip` returns.

Each iteration of the loop:

- Unpacks one tuple from the zipped items into three variables representing the `Axes` object, image and target value.
- Calls the `Axes` object's `imshow` method to display one image. The keyword argument `cmap=plt.cm.gray_r` determines the colors displayed in the image. The value `plt.cm.gray_r` is a **color map**—a group of colors that are typically chosen to work well together. This particular color map enables us to display the image's pixels in grayscale, with 0 as white, 16 as black and the values in between as gradually darkening shades of gray. For Matplotlib's color map names see https://matplotlib.org/examples/color/colormaps_reference.html. Each can be accessed through the `plt.cm` object or via a string, like `'gray_r'`.
- Calls the `Axes` object's `set_xticks` and `set_yticks` methods with empty lists to indicate that the *x*- and *y*-axes should not have tick marks.
- Calls the `Axes` object's `set_title` method to display the target value above the image—this shows the actual value that the image represents.

After the loop, we call `tight_layout` to remove the extra whitespace at the Figure's top, right, bottom and left, so the rows and columns of digit images can fill more of the Figure.

14.2.4 Splitting the Data for Training and Testing

You typically train a machine-learning model with a subset of a dataset. Typically, the more data you have for training, the better you can train the model. It's important to set aside a portion of your data for testing, so you can evaluate a model's performance using data that the model has not yet seen. Once you're confident that the model is performing well, you can use it to make predictions using new data it hasn't seen.

We first break the data into a **training set** and a **testing set** to prepare to train and test the model. The function `train_test_split` from the `sklearn.model_selection` module *shuffles* the data to randomize it, then splits the samples in the `data` array and the target values in the `target` array into training and testing sets. This helps ensure that the training and testing sets have similar characteristics. The shuffling and splitting is performed conveniently for you by a **ShuffleSplit** object from the `sklearn.model_selection` module. Function `train_test_split` returns a tuple of four elements in which the first two are the *samples* split into training and testing sets, and the last two are the corresponding *target values* split into training and testing sets. By convention, uppercase `X` is used to represent the samples, and lowercase `y` is used to represent the target values:

[lick here to view code image](#)

```
In [12]: from sklearn.model_selection import train_test_split

In [13]: X_train, X_test, y_train, y_test = train_test_split(
...:     digits.data, digits.target, random_state=11)
...:
```

We assume the data has **balanced classes**—that is, the samples are divided evenly among the classes. This is the case for each of scikit-learn’s bundled classification datasets. Unbalanced classes could lead to incorrect results.

In the “Functions” chapter, you saw how to *seed* a random-number generator for *reproducibility*. In machine-learning studies, this helps others confirm your results by working with the *same* randomly selected data. Function `train_test_split` provides the keyword argument `random_state` for *reproducibility*. When you run the code in the future with the *same* seed value, `train_test_split` will select the *same* data for the training set and the *same* data for the testing set. We chose the seed value (11) arbitrarily.

Training and Testing Set Sizes

Looking at `X_train`’s and `X_test`’s shapes, you can see that, *by default*, `train_test_split` reserves 75% of the data for training and 25% for testing:

[lick here to view code image](#)

```
In [14]: X_train.shape
Out[14]: (1347, 64)
```

```
In [15]: X_test.shape
Out[15]: (450, 64)
```

To specify *different* splits, you can set the sizes of the testing and training sets with the `train_test_split` function's keyword arguments `test_size` and `train_size`. Use floating-point values from 0.0 through 1.0 to specify the percentages of the data to use for each. You can use integer values to set the precise numbers of samples. If you specify one of these keyword arguments, the other is inferred. For example, the statement

[lick here to view code image](#)

```
X_train, X_test, y_train, y_test = train_test_split(
    digits.data, digits.target, random_state=11, test_size=0.20)
```

specifies that 20% of the data is for testing, so `train_size` is inferred to be 0.80.

14.2.5 Creating the Model

The `KNeighborsClassifier` estimator (module `sklearn.neighbors`) implements the k-nearest neighbors algorithm. First, we create the `KNeighborsClassifier` estimator object:

[lick here to view code image](#)

```
In [16]: from sklearn.neighbors import KNeighborsClassifier

In [17]: knn = KNeighborsClassifier()
```

To create an estimator, you simply create an object. The internal details of how this object implements the k-nearest neighbors algorithm are hidden in the object. You'll simply call its methods. This is the essence of Python *object-based programming*.

14.2.6 Training the Model

Next, we invoke the `KNeighborsClassifier` object's **`fit` method**, which loads the sample training set (`X_train`) and target training set (`y_train`) into the estimator:

[lick here to view code image](#)


```
In [18]: knn.fit(X=X_train, y=y_train)
Out[18]:
KNeighborsClassifier(algorithm='auto', leaf_size=30, metric='minkowski',
                    metric_params=None, n_jobs=None, n_neighbors=5, p=2,
                    weights='uniform')
```

For most, scikit-learn estimators, the `fit` method loads the data into the estimator then uses that data to perform complex calculations behind the scenes that learn from the data and train the model. The `KNeighborsClassifier`'s `fit` method just loads the data into the estimator, because k-NN actually has no initial learning process. The estimator is said to be **lazy** because its work is performed only when you use it to make predictions. In this and the next chapter, you'll use lots of models that have significant training phases. In the real-world machine-learning applications, it can sometimes take minutes, hours, days or even months to train your models. We'll see in the next chapter, "Deep Learning," that special-purpose, high-performance hardware called GPUs and TPUs can significantly reduce model training time.

As shown in snippet [18]'s output, the `fit` method returns the estimator, so IPython displays its string representation, which includes the estimator's *default* settings. The `n_neighbors` value corresponds to k in the k-nearest neighbors algorithm. By default, a `KNeighborsClassifier` looks at the five nearest neighbors to make its predictions. For simplicity, we generally use the default estimator settings. For `KNeighborsClassifier`, these are described at:

<http://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html>

Many of these settings are beyond the scope of this book. In Part 2 of this case study, we'll discuss how to choose the best value for `n_neighbors`.

14.2.7 Predicting Digit Classes

Now that we've loaded the data into the `KNeighborsClassifier`, we can use it with the test samples to make predictions. Calling the estimator's **predict method** with `X_test` as an argument returns an array containing the predicted class of each test image:

[lick here to view code image](#)

```
In [19]: predicted = knn.predict(X=X_test)
```

```
In [20]: expected = y_test
```

Let's look at the predicted digits vs. expected digits for the first 20 test samples:

[lick here to view code image](#)

```
In [21]: predicted[:20]
Out[21]: array([0, 4, 9, 9, 3, 1, 4, 1, 5, 0, 4, 9, 4, 1, 5, 3, 3, 8, 5, 6

n [22]: expected[:20]
Out[22]: array([0, 4, 9, 9, 3, 1, 4, 1, 5, 0, 4, 9, 4, 1, 5, 3, 3, 8, 3, 6
```

As you can see, in the first 20 elements, only the predicted and expected arrays' values at index 18 do not match. We expected a 3, but the model predicted a 5.

Let's use a list comprehension to locate *all* the incorrect predictions for the *entire* test set—that is, the cases in which the predicted and expected values do *not* match:

[lick here to view code image](#)

```
In [23]: wrong = [(p, e) for (p, e) in zip(predicted, expected) if p !=

n [24]: wrong
Out[24]:
[(5, 3),
 (8, 9),
 (4, 9),
 (7, 3),
 (7, 4),
 (2, 8),
 (9, 8),
 (3, 8),
 (3, 8),
 (1, 8)]
```

The list comprehension uses `zip` to create tuples containing the corresponding elements in `predicted` and `expected`. We include a tuple in the result only if its `p` (the predicted value) and `e` (the expected value) differ—that is, the predicted value was incorrect. In this example, the estimator incorrectly predicted only 10 of the 450 test samples. So the prediction accuracy of this estimator is an impressive 97.78%, even though we used only the estimator's default parameters.

14.3 CASE STUDY: CLASSIFICATION WITH K-NEAREST NEIGHBORS AND THE DIGITS DATASET, PART 2

In this section, we continue the digit classification case study. We'll:

- evaluate the k-NN classification estimator's accuracy,
- execute multiple estimators and can compare their results so you can choose the best one(s), and
- show how to tune k-NN's hyperparameter k to get the best performance out of a `KNeighborsClassifier`.

14.3.1 Metrics for Model Accuracy

Once you've trained and tested a model, you'll want to measure its accuracy. Here, we'll look at two ways of doing this—a classification estimator's `score` method and a *confusion matrix*.

Estimator Method `score`

Each estimator has a `score` method that returns an indication of how well the estimator performs for the test data you pass as arguments. For classification estimators, this method returns the *prediction accuracy* for the test data:

[lick here to view code image](#)

```
In [25]: print(f'{knn.score(X_test, y_test):.2%}')
97.78%
```

The `kNeighborsClassifier`'s with its default k (that is, `n_neighbors=5`) achieved 97.78% prediction accuracy. Shortly, we'll perform hyperparameter tuning to try to determine the optimal value for k , hoping that we get even better accuracy.

Confusion Matrix

Another way to check a classification estimator's accuracy is via a **confusion matrix**, which shows the correct and incorrect predicted values (also known as the *hits* and *misses*) for a given class. Simply call the function `confusion_matrix` from the **`sklearn.metrics` module**, passing the expected classes and the predicted classes as arguments, as in:

[lick here to view code image](#)

```
In [26]: from sklearn.metrics import confusion_matrix

In [27]: confusion = confusion_matrix(y_true=expected, y_pred=predicted)
```

The `y_true` keyword argument specifies the test samples' actual classes. People looked at the dataset's images and labeled them with specific classes (the digit values). The `y_pred` keyword argument specifies the predicted digits for those test images.

Below is the confusion matrix produced by the preceding call. The correct predictions are shown on the diagonal from top-left to bottom-right. This is called the **principal diagonal**. The nonzero values that are not on the principal diagonal indicate incorrect predictions:

[lick here to view code image](#)

```
In [28]: confusion
Out[28]:
array([[45,  0,  0,  0,  0,  0,  0,  0,  0,  0],
       [ 0, 45,  0,  0,  0,  0,  0,  0,  0,  0],
       [ 0,  0, 54,  0,  0,  0,  0,  0,  0,  0],
       [ 0,  0,  0, 42,  0,  1,  0,  1,  0,  0],
       [ 0,  0,  0,  0, 49,  0,  0,  1,  0,  0],
       [ 0,  0,  0,  0,  0, 38,  0,  0,  0,  0],
       [ 0,  0,  0,  0,  0,  0, 42,  0,  0,  0],
       [ 0,  0,  0,  0,  0,  0,  0, 45,  0,  0],
       [ 0,  1,  1,  2,  0,  0,  0,  0, 39,  1],
       [ 0,  0,  0,  0,  1,  0,  0,  0,  1, 41]])
```

Each row represents one distinct class—that is, one of the digits 0–9. The columns within a row specify how many of the test samples were classified into each distinct class. For example, row 0:

[lick here to view code image](#)

```
[45,  0,  0,  0,  0,  0,  0,  0,  0,  0]
```

represents the digit 0 class. The columns represent the ten possible target classes 0 through 9. Because we're working with digits, the classes (0–9) and the row and column index numbers (0–9) happen to match. According to row 0, 45 test samples

were classified as the digit 0, and *none* of the test samples were misclassified as any of the digits 1 through 9. So 100% of the 0s were correctly predicted.

On the other hand, consider row 8 which represents the results for the digit 8:

[lick here to view code image](#)

```
[ 0,  1,  1,  2,  0,  0,  0,  0, 39,  1]
```

- The 1 at column index 1 indicates that one 8 was *incorrectly* classified as a 1.
- The 1 at column index 2 indicates that one 8 was *incorrectly* classified as a 2.
- The 2 at column index 3 indicates that two 8s were *incorrectly* classified as 3s.
- The 39 at column index 8 indicates that 39 8s were *correctly* classified as 8s.
- The 1 at column index 9 indicates that one 8 was *incorrectly* classified as a 9.

So the algorithm correctly predicted 88.63% (39 of 44) of the 8s. Earlier we saw that the overall prediction accuracy of this estimator was 97.78%. The lower prediction accuracy for 8s indicates that they're apparently harder to recognize than the other digits.

Classification Report

The `sklearn.metrics` module also provides function `classification_report`, which produces a table of **classification metrics** ⁵ based on the expected and predicted values:

⁵ http://scikit-learn.org/stable/modules/model_evaluation.html#precision-recall-and-f-measures.

[lick here to view code image](#)

```
In [29]: from sklearn.metrics import classification_report

In [30]: names = [str(digit) for digit in digits.target_names]

In [31]: print(classification_report(expected, predicted,
...:                                target_names=names))
```

...	precision	recall	f1-score	support
0	1.00	1.00	1.00	45
1	0.98	1.00	0.99	45
2	0.98	1.00	0.99	54
3	0.95	0.95	0.95	44
4	0.98	0.98	0.98	50
5	0.97	1.00	0.99	38
6	1.00	1.00	1.00	42
7	0.96	1.00	0.98	45
8	0.97	0.89	0.93	44
9	0.98	0.95	0.96	43
micro avg	0.98	0.98	0.98	450
macro avg	0.98	0.98	0.98	450
weighted avg	0.98	0.98	0.98	450

In the report:

- **precision** is the total number of correct predictions for a given digit divided by the total number of predictions for that digit. You can confirm the precision by looking at each column in the confusion matrix. For example, if you look at column index 7, you'll see 1s in rows 3 and 4, indicating that one 3 and one 4 were incorrectly classified as 7s and a 45 in row 7 indicating the 45 images were correctly classified as 7s. So the *precision* for the digit 7 is 45/47 or 0.96.
- **recall** is the total number of correct predictions for a given digit divided by the total number of samples that should have been predicted as that digit. You can confirm the recall by looking at each row in the confusion matrix. For example, if you look at row index 8, you'll see three 1s and a 2 indicating that some 8s were incorrectly classified as other digits and a 39 indicating that 39 images were correctly classified. So the *recall* for the digit 8 is 39/44 or 0.89.
- **f1-score**—This is the average of the *precision* and the *recall*.
- **support**—The number of samples with a given expected value. For example, 50 samples were labeled as 4s, and 38 samples were labeled as 5s.

For details on the averages displayed at the bottom of the report, see:

http://scikit-learn.org/stable/modules/generated/sklearn.metrics.classification_report.html

Visualizing the Confusion Matrix

A **heat map** displays values as colors, often with values of higher magnitude displayed as more intense colors. Seaborn's graphing functions work with two-dimensional data. When using a pandas `DataFrame` as the data source, Seaborn automatically labels its visualizations using the column names and row indices. Let's convert the confusion matrix into a `DataFrame`, then graph it:

[lick here to view code image](#)

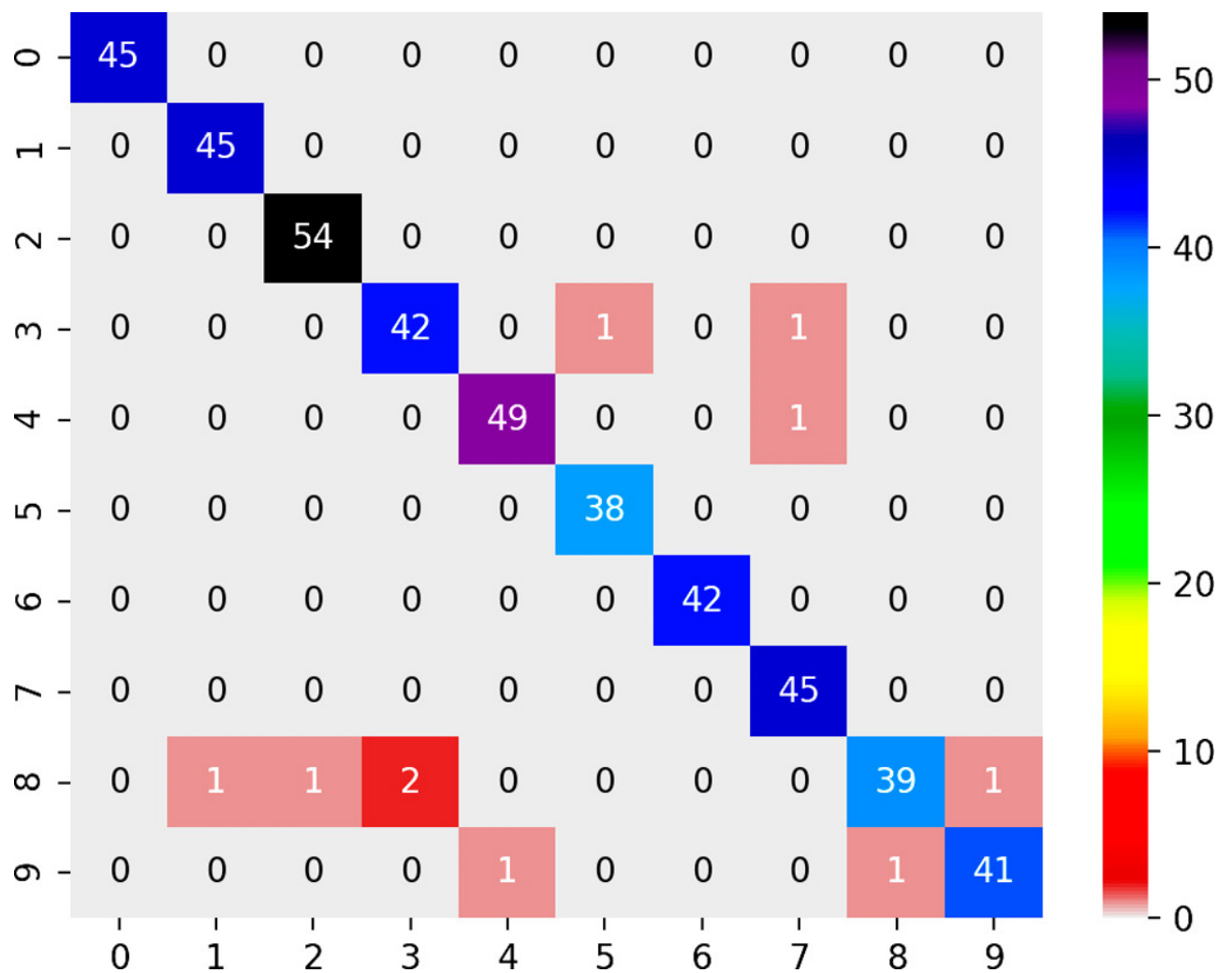
```
In [32]: import pandas as pd

In [33]: confusion_df = pd.DataFrame(confusion,    index=range(10),
...:                                columns=range(10))
...:

In [34]: import seaborn as sns

In [35]: axes = sns.heatmap(confusion_df, annot=True,
...:                        cmap='nipy_spectral_r')
...:
```

The Seaborn function **heatmap** creates a heat map from the specified `DataFrame`. The keyword argument `annot=True` (short for “annotation”) displays a color bar to the right of the diagram, showing how the values correspond to the heat map's colors. The `cmap='nipy_spectral_r'` keyword argument specifies which color map to use. We used the `nipy_spectral_r` color map with the colors shown in the heat map's color bar. When you display a confusion matrix as a heat map, the principal diagonal and the incorrect predictions stand out nicely.



14.3.2 K-Fold Cross-Validation

K-fold cross-validation enables you to use all of your data for *both* training *and* testing, to get a better sense of how well your model will make predictions for new data by repeatedly training and testing the model with different portions of the dataset. K-fold cross-validation splits the dataset into k equal-size **folds** (this k is unrelated to k in the k-nearest neighbors algorithm). You then repeatedly train your model with $k - 1$ folds and test the model with the remaining fold. For example, consider using $k = 10$ with folds numbered 1 through 10. With 10 folds, we'd do 10 successive training and testing cycles:

- First, we'd train with folds 1–9, then test with fold 10.
- Next, we'd train with folds 1–8 and 10, then test with fold 9.
- Next, we'd train with folds 1–7 and 9–10, then test with fold 8.

This training and testing cycle continues until each fold has been used to test the model.

KFold Class

Scikit-learn provides the **KFold** class and the **cross_val_score** function (both in the module `sklearn.model_selection`) to help you perform the training and testing cycles described above. Let's perform k-fold cross-validation with the Digits dataset and the `KNeighborsClassifier` created earlier. First, create a `KFold` object:

[lick here to view code image](#)

```
In [36]: from sklearn.model_selection import KFold

In [37]: kfold = KFold(n_splits=10,    random_state=11, shuffle=True)
```

The keyword arguments are:

- `n_splits=10`, which specifies the number of folds.
- `random_state=11`, which seeds the random number generator for *reproducibility*.
- `shuffle=True`, which causes the `KFold` object to randomize the data by shuffling it before splitting it into folds. This is particularly important if the samples might be ordered or grouped. For example, the Iris dataset we'll use later in this chapter has 150 samples of three *Iris* species—the first 50 are *Iris setosa*, the next 50 are *Iris versicolor* and the last 50 are *Iris virginica*. If we do not shuffle the samples, then the training data might contain none of a particular *Iris* species and the test data might be all of one species.

Using the KFold Object with Function `cross_val_score`

Next, use function `cross_val_score` to train and test your model:

[lick here to view code image](#)

```
In [38]: from sklearn.model_selection import cross_val_score

In [39]: scores = cross_val_score(estimator=knn,    X=digits.data,
...:                               y=digits.target, cv=kfold)
...:
```

The keyword arguments are:

- `estimator=knn`, which specifies the estimator you'd like to validate.
- `X=digits.data`, which specifies the samples to use for training and testing.
- `y=digits.target`, which specifies the target predictions for the samples.
- `cv=kfold`, which specifies the cross-validation generator that defines how to split the samples and targets for training and testing.

Function `cross_val_score` returns an array of accuracy scores—one for each fold. As you can see below, the model was quite accurate. Its *lowest* accuracy score was `0.97777778` (97.78%) and in one case it was 100% accurate in predicting an entire fold:

[lick here to view code image](#)

```
In [40]: scores
Out[40]:
array([0.97777778, 0.99444444, 0.98888889, 0.97777778, 0.98888889,
       0.99444444, 0.97777778, 0.98882682, 1.          , 0.98324022])
```

Once you have the accuracy scores, you can get an overall sense of the model's accuracy by calculating the mean accuracy score and the standard deviation among the 10 accuracy scores (or whatever number of folds you choose):

[lick here to view code image](#)

```
In [41]: print(f'Mean accuracy: {scores.mean():.2%}')
Mean accuracy: 98.72%

In [42]: print(f'Accuracy standard deviation: {scores.std():.2%}')
Accuracy standard deviation: 0.75%
```

On average, the model was 98.72% accurate—even better than the 97.78% we achieved when we trained the model with 75% of the data and tested the model with 25% earlier.

14.3.3 Running Multiple Models to Find the Best One

It's difficult to know in advance which machine learning model(s) will perform best for a given dataset, especially when they hide the details of how they operate from their users. Even though the `KNeighborsClassifier` predicts digit images with a high

degree of accuracy, it's possible that other scikit-learn estimators are even more accurate. Scikit-learn provides many models with which you can quickly train and test your data. This encourages you to run *multiple models* to determine which is the best for a particular machine learning study.

Let's use the techniques from the preceding section to compare several classification estimators—`KNeighborsClassifier`, `SVC` and `GaussianNB` (there are more). Though we have not studied the `SVC` and `GaussianNB` estimators, scikit-learn nevertheless makes it easy for you to test-drive them by using their default settings.⁶ First, let's import the other two estimators:

⁶ To avoid a warning in the current scikit-learn version at the time of this writing (version 0.20), we supplied one keyword argument when creating the `SVC` estimator. This arguments value will become the default in scikit-learn version 0.22.

[lick here to view code image](#)

```
In [43]: from sklearn.svm import SVC

In [44]: from sklearn.naive_bayes import GaussianNB
```

Next, let's create the estimators. The following dictionary contains key–value pairs for the existing `KNeighborsClassifier` we created earlier, plus new `SVC` and `GaussianNB` estimators:

[lick here to view code image](#)

```
In [45]: estimators = {
...:     'KNeighborsClassifier': knn,
...:     'SVC': SVC(gamma='scale'),
...:     'GaussianNB': GaussianNB() }
...:
```

Now, we can execute the models:

[lick here to view code image](#)

```
In [46]: for estimator_name, estimator_object in estimators.items():
...:     kfold = KFold(n_splits=10, random_state=11, shuffle=True)
...:     scores = cross_val_score(estimator=estimator_object,
...:                               X=digits.data, y=digits.target, cv=kfold)
...:     print(f'{estimator_name:>20}: ' +
```

```

...:         f'mean accuracy={scores.mean():.2%}; ' +
...:         f'standard deviation={scores.std():.2%}')
...:
KNeighborsClassifier: mean accuracy=98.72%; standard deviation=0.75%
SVC: mean accuracy=99.00%; standard deviation=0.85%
GaussianNB: mean accuracy=84.48%; standard deviation=3.47%

```

This loop iterates through items in the `estimators` dictionary and for each key-value pair performs the following tasks:

- Unpacks the key into `estimator_name` and value into `estimator_object`.
- Creates a `KFold` object that shuffles the data and produces 10 folds. The keyword argument `random_state` is particularly important here because it ensures that each estimator works with identical folds, so we’re comparing “apples to apples.”
- Evaluates the current `estimator_object` using `cross_val_score`.
- Prints the estimator’s name, followed by the mean and standard deviation of the accuracy scores’ computed for each of the 10 folds.

Based on the results, it appears that we can get slightly better accuracy from the `SVC` estimator—at least when using the estimator’s default settings. It’s possible that by tuning some of the estimators’ settings, we could get even better results. The `KNeighborsClassifier` and `SVC` estimators’ accuracies are nearly identical so we might want to perform hyperparameter tuning on each to determine the best.

Scikit-Learn Estimator Diagram

The scikit-learn documentation provides a helpful diagram for choosing the right estimator, based on the kind and size of your data and the machine learning task you wish to perform:

https://scikit-learn.org/stable/tutorial/machine_learning_map/index.html

14.3.4 Hyperparameter Tuning

Earlier in this section, we mentioned that k in the k-nearest neighbors algorithm is a hyperparameter of the algorithm. Hyperparameters are set *before* using the algorithm to train your model. In real-world machine learning studies, you’ll want to use hyperparameter tuning to choose hyperparameter values that produce the best possible

predictions.

To determine the best value for k in the kNN algorithm, try different values of k then compare the estimator's performance with each. We can do this using techniques similar to comparing estimators. The following loop creates `KNeighborsClassifiers` with odd k values from 1 through 19 (again, we use odd k values in kNN to avoid ties) and performs k -fold cross-validation on each. As you can see from the accuracy scores and standard deviations, the k value 1 in kNN produces the most accurate predictions for the Digits dataset. You can also see that accuracy tends to decrease for higher k values:

[lick here to view code image](#)

```
In [47]: for k in range(1, 20, 2):
...:     kfold = KFold(n_splits=10, random_state=11, shuffle=True)
...:     knn = KNeighborsClassifier(n_neighbors=k)
...:     scores = cross_val_score(estimator=knn,
...:                             X=digits.data, y=digits.target, cv=kfold)
...:     print(f'k={k}<2>; mean accuracy={scores.mean():.2%}; ' +
...:           f'standard deviation={scores.std():.2%}')
...:
k=1 ; mean accuracy=98.83%; standard deviation=0.58%
k=3 ; mean accuracy=98.78%; standard deviation=0.78%
k=5 ; mean accuracy=98.72%; standard deviation=0.75%
k=7 ; mean accuracy=98.44%; standard deviation=0.96%
k=9 ; mean accuracy=98.39%; standard deviation=0.80%
k=11; mean accuracy=98.39%; standard deviation=0.80%
k=13; mean accuracy=97.89%; standard deviation=0.89%
k=15; mean accuracy=97.89%; standard deviation=1.02%
k=17; mean accuracy=97.50%; standard deviation=1.00%
k=19; mean accuracy=97.66%; standard deviation=0.96%
```

Machine learning is not without its costs, especially as we head toward big data and deep learning. You must “know your data” and “know your tools.” For example, compute time grows rapidly with k , because k -NN needs to perform more calculations to find the nearest neighbors. There is also function `cross_validate`, which does cross-validation *and* times the results.

14.4 CASE STUDY: TIME SERIES AND SIMPLE LINEAR REGRESSION

In the previous section, we demonstrated classification in which each sample was associated with a *distinct* class. Here, we continue our discussion of simple linear

regression—the simplest of the regression algorithms—that began in [chapter 10](#)’s Intro to Data Science section. Recall that given a collection of numeric values representing an independent variable and a dependent variable, simple linear regression describes the relationship between these variables with a straight line, known as the regression line.

Previously, we performed simple linear regression on a time series of average New York City January high-temperature data for 1895 through 2018. In that example, we used Seaborn’s `regplot` function to create a scatter plot of the data with a corresponding regression line. We also used the `scipy.stats` module’s `linregress` function to calculate the regression line’s slope and intercept. We then used those values to predict future temperatures and estimate past temperatures.

In this section, we’ll

- use a *scikit-learn estimator* to reimplement the simple linear regression we showed in [chapter 10](#),
- use Seaborn’s `scatterplot` function to plot the data and Matplotlib’s `plot` function to display the regression line, then
- use the coefficient and intercept values calculated by the scikit-learn estimator to make predictions.

Later, we’ll look at *multiple linear regression* (also simply called *linear regression*).

For your convenience, we provide the temperature data in the `ch14` examples folder in a CSV file named `ave_hi_nyc_jan_1895-2018.csv`. Once again, launch IPython with the `--matplotlib` option:

```
ipython --matplotlib
```

Loading the Average High Temperatures into a `DataFrame`

As we did in [chapter 10](#), let’s load the data from `ave_hi_nyc_jan_1895-2018.csv`, rename the `'Value'` column to `'Temperature'`, remove `01` from the end of each date value and display a few data samples:

[lick here to view code image](#)

```
In [1]: import pandas as pd
```

```
In [2]: nyc = pd.read_csv('ave_hi_nyc_jan_1895-2018.csv')

In [3]: nyc.columns = ['Date', 'Temperature', 'Anomaly']

In [4]: nyc.Date = nyc.Date.floordiv(100)

In [5]: nyc.head(3)
Out[5]:
```

	Date	Temperature	Anomaly
0	1895	34.2	-3.2
1	1896	34.7	-2.7
2	1897	35.5	-1.9

Splitting the Data for Training and Testing

In this example, we'll use the **LinearRegression** estimator from **sklearn.linear_model**. By default, this estimator uses *all* the numeric features in a dataset, performing a **multiple linear regression** (which we'll discuss in the next section). Here, we perform *simple linear regression* using *one* feature as the independent variable. So, we'll need to select one feature (the `Date`) from the dataset.

When you select one column from a two-dimensional `DataFrame`, the result is a *one-dimensional Series*. However, scikit-learn estimators require their training and testing data to be *two-dimensional arrays* (or two-dimensional *array-like* data, such as lists of lists or pandas `DataFrames`). To use one-dimensional data with an estimator, you must transform it from one dimension containing n elements, into two dimensions containing n rows and one *column* as you'll see below.

As we did in the previous case study, let's split the data into training and testing sets. Once again, we used the keyword argument `random_state` for reproducibility:

[lick here to view code image](#)

```
In [6]: from sklearn.model_selection import train_test_split

In [7]: X_train, X_test, y_train, y_test = train_test_split(
...:     nyc.Date.values.reshape(-1, 1), nyc.Temperature.values,
...:     random_state=11)
...:
```

The expression `nyc.Date` returns the `Date` column's `Series`, and the `Series`' `values` attribute returns the NumPy array containing that `Series`' values. To transform this one-dimensional array into two dimensions, we call the array's **reshape**

method. Normally, two arguments are the precise number of rows and columns. However, the first argument `-1` tells `reshape` to *infer* the number of rows, based on the number of columns (1) and the number of elements (124) in the array. The transformed array will have only one column, so `reshape` infers the number of rows to be 124, because the only way to fit 124 elements into an array with one column is by distributing them over 124 rows.

We can confirm the 75%–25% train-test split by checking the shapes of `X_train` and `X_test`:

[lick here to view code image](#)

```
In [8]: X_train.shape
Out[8]: (93, 1)

In [9]: X_test.shape
Out[9]: (31, 1)
```

Training the Model

Scikit-learn does not have a separate class for simple linear regression because it's just a special case of multiple linear regression, so let's train a `LinearRegression` estimator:

[lick here to view code image](#)

```
In [10]: from sklearn.linear_model import LinearRegression

In [11]: linear_regression = LinearRegression()

In [12]: linear_regression.fit(X=X_train, y=y_train)
Out[12]:
LinearRegression(copy_X=True, fit_intercept=True, n_jobs=None,
                  normalize=False)
```

After training the estimator, `fit` returns the estimator, and IPython displays its string representation. For descriptions of the default settings, see:

http://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html

To find the best fitting regression line for the data, the `LinearRegression` estimator iteratively adjusts the slope and intercept values to minimize the sum of the squares of

the data points' distances from the line. In [chapter 10's Intro to Data Science](#) section, we gave some insight into how the slope and intercept values are discovered.

Now, we can get the slope and intercept used in the $y = mx + b$ calculation to make predictions. The slope is stored in the estimator's `coeff_` attribute (m in the equation) and the intercept is stored in the estimator's `intercept_` attribute (b in the equation):

[lick here to view code image](#)

```
In [13]: linear_regression.coef_  
Out[13]: array([0.01939167])  
  
In [14]: linear_regression.intercept_  
Out[14]: -0.30779820252656265
```

We'll use these later to plot the regression line and make predictions for specific dates.

Testing the Model

Let's test the model using the data in `X_test` and check some of the predictions throughout the dataset by displaying the predicted and expected values for every fifth element—we discuss how to assess the regression model's accuracy in [section 4.5.8](#):

[lick here to view code image](#)

```
In [15]: predicted = linear_regression.predict(X_test)  
  
In [16]: expected = y_test  
  
In [17]: for p, e in zip(predicted[::5], expected[::5]):  
...:     print(f'predicted: {p:.2f}, expected: {e:.2f}')  
...:  
predicted: 37.86, expected: 31.70  
predicted: 38.69, expected: 34.80  
predicted: 37.00, expected: 39.40  
predicted: 37.25, expected: 45.70  
predicted: 38.05, expected: 32.30  
predicted: 37.64, expected: 33.80  
predicted: 36.94, expected: 39.70
```

Predicting Future Temperatures and Estimating Past Temperatures

Let's use the coefficient and intercept values to predict the January 2019 average high temperature and to estimate what the average high temperature was in January of

1890. The `lambda` in the following snippet implements the equation for a line

$$y = mx + b$$

using the `coef_` as m and the `intercept_` as b .

[lick here to view code image](#)

```
In [18]: predict = (lambda x: linear_regression.coef_ * x +
...:                linear_regression.intercept_)
...:

In [19]: predict(2019)
Out[19]: array([38.84399018])

In [20]: predict(1890)
Out[20]: array([36.34246432])
```

Visualizing the Dataset with the Regression Line

Next, let's create a scatter plot of the dataset using Seaborn's `scatterplot` function and Matplotlib's `plot` function. First, use `scatterplot` with the `nyc` DataFrame to display the data points:

[lick here to view code image](#)

```
In [21]: import seaborn as sns

In [22]: axes = sns.scatterplot(data=nyc, x='Date', y='Temperature',
...:                             hue='Temperature', palette='winter', legend=False)
...:
```

The keyword arguments are:

- `data`, which specifies the DataFrame (`nyc`) containing the data to display.
- `x` and `y`, which specify the names of `nyc`'s columns that are the source of the data along the x - and y -axes, respectively. In this case, `x` is the 'Date' and `y` is the 'Temperature'. The corresponding values from each column form x - y coordinate pairs used to plot the dots.
- `hue`, which specifies which column's data should be used to determine the dot

colors. In this case, we use the 'Temperature' column. Color is not particularly important in this example, but we wanted to add some visual interest to the graph.

- `palette`, which specifies a Matplotlib color map from which to choose the dots' colors.
- `legend=False`, which specifies that `scatterplot` should not show a legend for the graph—the default is `True`, but we do not need a legend for this example.

As we did in chapter 10, let's scale the y-axis range of values so you'll be able to see the linear relationship better once we display the regression line:

[lick here to view code image](#)

```
In [23]: axes.set_ylim(10, 70)
Out[23]: (10, 70)
```

Next, let's display the regression line. First, create an array containing the minimum and maximum date values in `nyc.Date`. These are the x-coordinates of the regression line's start and end points:

[lick here to view code image](#)

```
In [24]: import numpy as np

In [25]: x = np.array([min(nyc.Date.values), max(nyc.Date.values)])
```

Passing the array `x` to the `predict` lambda from snippet [16] produces an array containing the corresponding predicted values, which we'll use as the y-coordinates:

[lick here to view code image](#)

```
In [26]: y = predict(x)
```

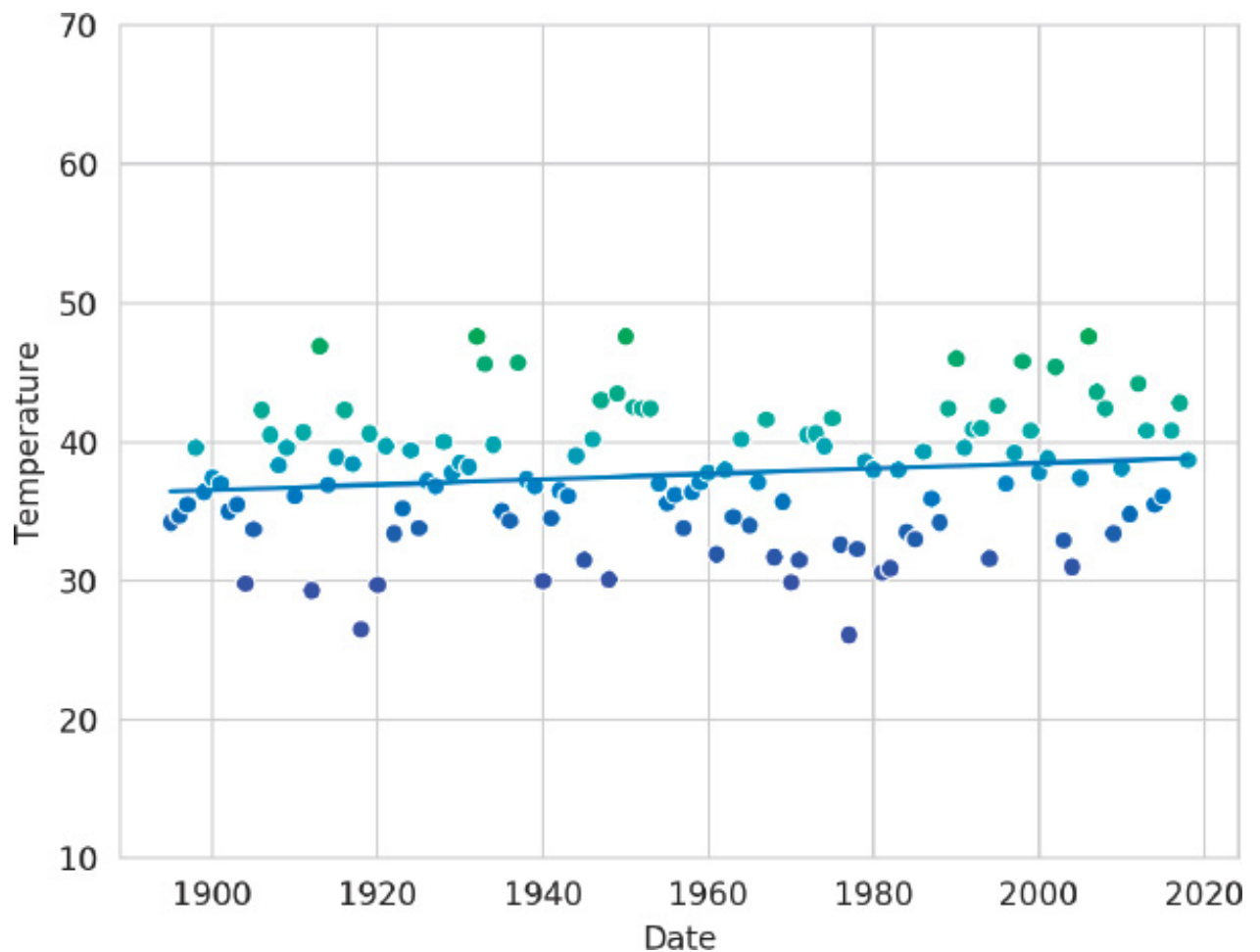
Finally, we can use Matplotlib's `plot` function to plot a line based on the `x` and `y` arrays, which represent the x- and y-coordinates of the points, respectively:

[lick here to view code image](#)

```
In [27]: import matplotlib.pyplot as plt
```

```
In [28]: line = plt.plot(x, y)
```

The resulting scatterplot and regression line are shown below. This graph is nearly identical to the one you saw in [chapter 10's Intro to Data Science section](#).



Overfitting/Underfitting

When creating a model, a key goal is to ensure that it is capable of making accurate predictions for data it has not yet seen. Two common problems that prevent accurate predictions are overfitting and underfitting:

- **Underfitting** occurs when a model is too simple to make predictions, based on its training data. For example, you may use a linear model, such as simple linear regression, when in fact, the problem really requires a non-linear model. For example, temperatures vary significantly throughout the four seasons. If you're trying to create a general model that can predict temperatures year-round, a simple linear regression model will underfit the data.
- **Overfitting** occurs when your model is too complex. The most extreme case, would be a model that memorizes its training data. That may be acceptable if your new data looks *exactly* like your training data, but ordinarily that's not the case. When

you make predictions with an overfit model, new data that matches the training data will produce perfect predictions, but the model will not know what to do with data it has never seen.

For additional information on underfitting and overfitting, see

- <https://en.wikipedia.org/wiki/Overfitting>
- <https://machinelearningmastery.com/overfitting-and-underfitting-with-machine-learning-algorithms/>

14.5 CASE STUDY: MULTIPLE LINEAR REGRESSION WITH THE CALIFORNIA HOUSING DATASET

In chapter 10's Intro to Data Science section, we performed simple linear regression on a small weather data time series using pandas, Seaborn's `regplot` function and the SciPy's `stats` module's `linregress` function. In the previous section, we reimplemented that same example using scikit-learn's `LinearRegression` estimator, Seaborn's `scatterplot` function and Matplotlib's `plot` function. Now, we'll perform linear regression with a much larger real-world dataset.

The **California Housing dataset**⁷ bundled with scikit-learn has 20,640 samples, each with eight numerical features. We'll perform a *multiple linear regression* that uses all eight numerical features to make more sophisticated housing price predictions than if we were to use only a single feature or a subset of the features. Once again, scikit-learn will do most of the work for you—`LinearRegression` performs multiple linear regression by default.

⁷ <http://lib.stat.cmu.edu/datasets>. Pace, R. Kelley and Ronald Barry, Sparse Spatial Autoregressions, Statistics and Probability Letters, 33 (1997) 291-297. Submitted to the StatLib Datasets Archive by Kelley Pace (pace@unix1.sncc.lsu.edu). [9/Nov/99].

We'll visualize some of the data using Matplotlib and Seaborn, so launch IPython with Matplotlib support:

```
ipython --matplotlib
```

14.5.1 Loading the Dataset

According to the California Housing Prices dataset's description in scikit-learn, "This dataset was derived from the 1990 U.S. census, using one row per census block group. A block group is the smallest geographical unit for which the U.S. Census Bureau publishes sample data (a block group typically has a population of 600 to 3,000 people)." The dataset has 20,640 samples—one per block group—with eight features each:

- median income—in tens of thousands, so 8.37 would represent \$83,700
- median house age—in the dataset, the maximum value for this feature is 52
- average number of rooms
- average number of bedrooms
- block population
- average house occupancy
- house block latitude
- house block longitude

Each sample also has as its *target* a corresponding median house value in hundreds of thousands, so 3.55 would represent \$355,000. In the dataset, the maximum value for this feature is 5, which represents \$500,000.

It's reasonable to expect that more bedrooms or more rooms or higher income would mean higher house value. By combining these features to make predictions, we're more likely to get more accurate predictions.

Loading the Data

Let's load the dataset and familiarize ourselves with it. The `fetch_california_housing` function from the `sklearn.datasets` module returns a `Bunch` object containing the data and other information about the dataset:

[lick here to view code image](#)

```
In [1]: from sklearn.datasets import fetch_california_housing

In [2]: california = fetch_california_housing()
```

Displaying the Dataset's Description

Let's look at the dataset's description. The `DESCR` information includes:

- Number of Instances—this dataset contains 20,640 samples.
- Number of Attributes—there are 8 features (attributes) per sample.
- Attribute Information—feature descriptions.
- Missing Attribute Values—none are missing in this dataset.

According to the description, the target variable in this dataset is the *median house value*—this is the value we'll be trying to predict via multiple linear regression.

[lick here to view code image](#)

```
n [3]: print(california.DESCR)
.. _california_housing_dataset:

California Housing dataset
-----

**Data Set Characteristics:**

:Number of Instances: 20640

:Number of Attributes: 8 numeric, predictive attributes and
the target

:Attribute Information:
  - MedInc           median income in block
  - HouseAge         median house age in block
  - AveRooms         average number of rooms
  - AveBedrms        average number of bedrooms
  - Population       block population
  - AveOccup         average house occupancy
  - Latitude         house block latitude
  - Longitude        house block longitude

:Missing Attribute Values: None

This dataset was obtained from the StatLib repository.
http://lib.stat.cmu.edu/datasets/

The target variable is the median house value for California districts.

This dataset was derived from the 1990 U.S. census, using one row per ce
```

```
It can be downloaded/loaded using the  
:func:`sklearn.datasets.fetch_california_housing` function.
```

```
.. topic:: References
```

- Pace, R. Kelley and Ronald Barry, Sparse Spatial Autoregressions, Statistics and Probability Letters, 33 (1997) 291-297

gain, the Bunch object's data and target attributes are NumPy arrays containing the 20,640 samples and their target values respectively. We can confirm the number of samples (rows) and features (columns) by looking at the data array's shape attribute, which shows that there are 20,640 rows and 8 columns:

[lick here to view code image](#)

```
In [4]: california.data.shape  
Out[4]: (20640, 8)
```

Similarly, you can see that the number of target values—that is, the median house values—matches the number of samples by looking at the target array's shape:

[lick here to view code image](#)

```
In [5]: california.target.shape  
Out[5]: (20640,)
```

The Bunch's **feature_names attribute** contains the names that correspond to each column in the data array:

[lick here to view code image](#)

```
In [6]: california.feature_names  
Out[6]:  
['MedInc',  
 'HouseAge',  
 'AveRooms',  
 'AveBedrms',  
 'Population',  
 'AveOccup',  
 'Latitude',  
 'Longitude']
```


14.5.2 Exploring the Data with Pandas

Let's use a pandas `DataFrame` to explore the data further. We'll also use the `DataFrame` with Seaborn in the next section to visualize some of the data. First, let's import pandas and set some options:

[lick here to view code image](#)

```
In [7]: import pandas as pd

In [8]: pd.set_option('precision', 4)

In [9]: pd.set_option('max_columns', 9)

In [10]: pd.set_option('display.width', None)
```

In the preceding `set_option` calls:

- `'precision'` is the maximum number of digits to display to the right of each decimal point.
- `'max_columns'` is the maximum number of columns to display when you output the `DataFrame`'s string representation. By default, if pandas cannot fit all of the columns left-to-right, it cuts out columns in the middle and displays an ellipsis () instead. The `'max_columns'` setting enables pandas to show all the columns using multiple rows of output. As you'll see momentarily, we'll have nine columns in the `DataFrame`—the eight dataset features in `california.data` and an additional column for the target median house values (`california.target`).
- `'display.width'` specifies the width in characters of your Command Prompt (Windows), Terminal (macOS/Linux) or shell (Linux). The value `None` tells pandas to auto-detect the display width when formatting string representations of `Series` and `DataFrames`.

Next, let's create a `DataFrame` from the Bunch's `data`, `target` and `feature_names` arrays. The first snippet below creates the initial `DataFrame` using the data in `california.data` and with the column names specified by `california.feature_names`. The second statement adds a column for the median house values stored in `california.target`:

[lick here to view code image](#)

```
In [11]: california_df = pd.DataFrame(california.data,
...:                                  columns=california.feature_names)
...:
In [12]: california_df['MedHouseValue'] = pd.Series(california.target)
```

We can peek at some of the data using the `head` function. Notice that pandas displays the `DataFrame`'s first six columns, then skips a line of output and displays the remaining columns. The `\` to the right of the column head "AveOccup" indicates that there are more columns displayed below. You'll see the `\` only if the window in which IPython is running is too narrow to display all the columns left-to-right:

[lick here to view code image](#)

```
In [13]: california_df.head()
Out[13]:
```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	\
0	8.3252	41.0	6.9841	1.0238	322.0	2.5556	
1	8.3014	21.0	6.2381	0.9719	2401.0	2.1098	
2	7.2574	52.0	8.2881	1.0734	496.0	2.8023	
3	5.6431	52.0	5.8174	1.0731	558.0	2.5479	
4	3.8462	52.0	6.2819	1.0811	565.0	2.1815	

	Latitude	Longitude	MedHouseValue
0	37.88	-122.23	4.526
1	37.86	-122.22	3.585
2	37.85	-122.24	3.521
3	37.85	-122.25	3.413
4	37.85	-122.25	3.422

Let's get a sense of the data in each column by calculating the `DataFrame`'s summary statistics. Note that the median income and house values (again, measured in hundreds of thousands) are from 1990 and are significantly higher today:

[lick here to view code image](#)

```
In [14]: california_df.describe()
Out[14]:
```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	\
count	20640.0000	20640.0000	20640.0000	20640.0000	20640.0000	
mean	3.8707	28.6395	5.4290	1.0967	1425.4767	
std	1.8998	12.5856	2.4742	0.4739	1132.4621	
min	0.4999	1.0000	0.8462	0.3333	3.0000	
25%	2.5634	18.0000	4.4407	1.0061	787.0000	
50%	3.5348	29.0000	5.2291	1.0488	1166.0000	

75%	4.7432	37.0000	6.0524	1.0995	1725.0000
max	15.0001	52.0000	141.9091	34.0667	35682.0000

	AveOccup	Latitude	Longitude	MedHouseValue
count	20640.0000	20640.0000	20640.0000	20640.0000
mean	3.0707	35.6319	-119.5697	2.0686
std	10.3860	2.1360	2.0035	1.1540
min	0.6923	32.5400	-124.3500	0.1500
25%	2.4297	33.9300	-121.8000	1.1960
50%	2.8181	34.2600	-118.4900	1.7970
75%	3.2823	37.7100	-118.0100	2.6472
max	1243.3333	41.9500	-114.3100	5.0000

14.5.3 Visualizing the Features

It's helpful to visualize your data by plotting the target value against *each* feature—in this case, to see how the median home value relates to each feature. To make our visualizations clearer, let's use `DataFrame` method **sample** to randomly select 10% of the 20,640 samples for graphing purposes:

[lick here to view code image](#)

```
In [15]: sample_df = california_df.sample(frac=0.1, random_state=17)
```

The keyword argument `frac` specifies the fraction of the data to select (0.1 for 10%), and the keyword argument `random_state` enables you to seed the random number generator. The integer seed value (17), which we chose arbitrarily, is crucial for *reproducibility*. Each time you use the *same* seed value, method `sample` selects the *same* random subset of the `DataFrame`'s rows. Then, when we graph the data, you should get the *same* results.

Next, we'll use Matplotlib and Seaborn to display scatter plots of each of the eight features. Both libraries can display scatter plots. Seaborn's are more attractive and require less code, so we'll use Seaborn to create the following scatter plots. First, we import both libraries and use Seaborn function `set` to scale each diagram's fonts to two times their default size:

[lick here to view code image](#)

```
In [16]: import matplotlib.pyplot as plt
```

```
In [17]: import seaborn as sns
```

```
In [18]: sns.set(font_scale=2)

In [19]: sns.set_style('whitegrid')
```

The following snippet displays the scatter plots.⁸ Each shows one feature along the x -axis and the median home value (`california.target`) along the y -axis, so we can see how each feature and the median house values relate to one another. We display each scatter plot in a separate window. The windows are displayed in the order the features were listed in snippet [6] with the most recently displayed window in the foreground:

⁸ When you execute this code in IPython, each window will be displayed in front of the previous one. As you close each, you'll see the one behind it.

[lick here to view code image](#)

```
In [20]: for feature in california.feature_names:
...:     plt.figure(figsize=(16, 9))
...:     sns.scatterplot(data=sample_df, x=feature,
...:                     y='MedHouseValue', hue='MedHouseValue',
...:                     palette='cool', legend=False)
...:
```

For each feature name, the snippet first creates a 16-inch-by-9-inch Matplotlib Figure—we're plotting many data points, so we chose to use a larger window. If this window is larger than your screen, Matplotlib fits the Figure to the screen. Seaborn uses the current Figure to display the scatter plot. If you do not create a Figure first, Seaborn will create one. We created the Figure first here so we could display a large window for a scatter plot containing over 2000 points.

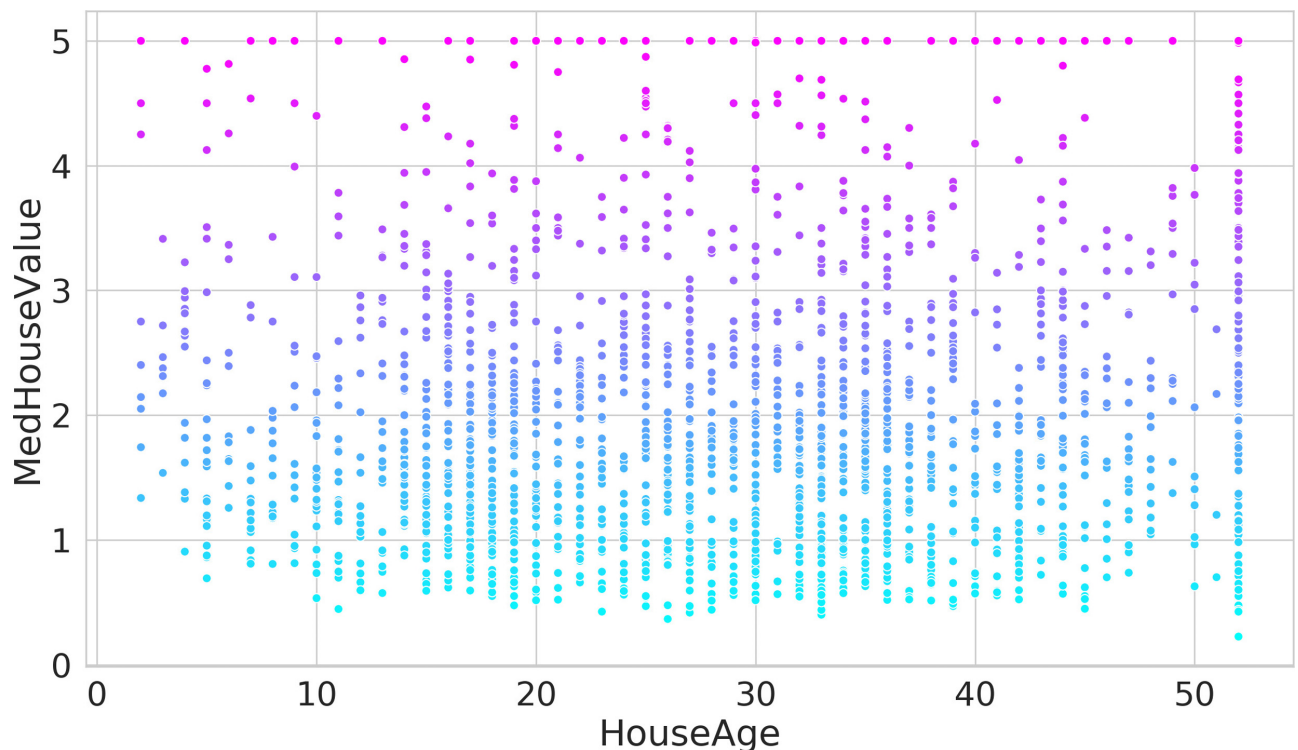
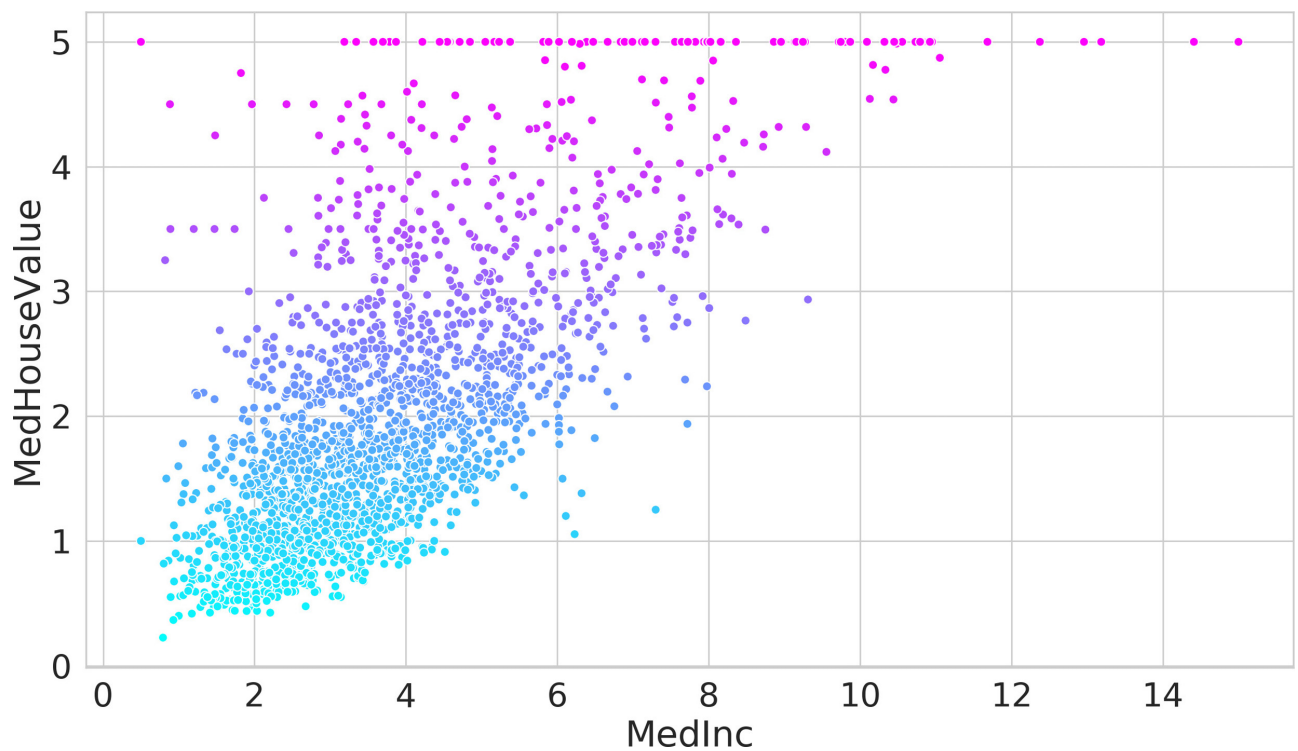
Next, the snippet creates a Seaborn `scatterplot` in which the x -axis shows the current feature, the y -axis shows the 'MedHouseValue' (*median house values*), and the 'MedHouseValue' determines the dot colors (*hue*). Some interesting things to notice in these graphs:

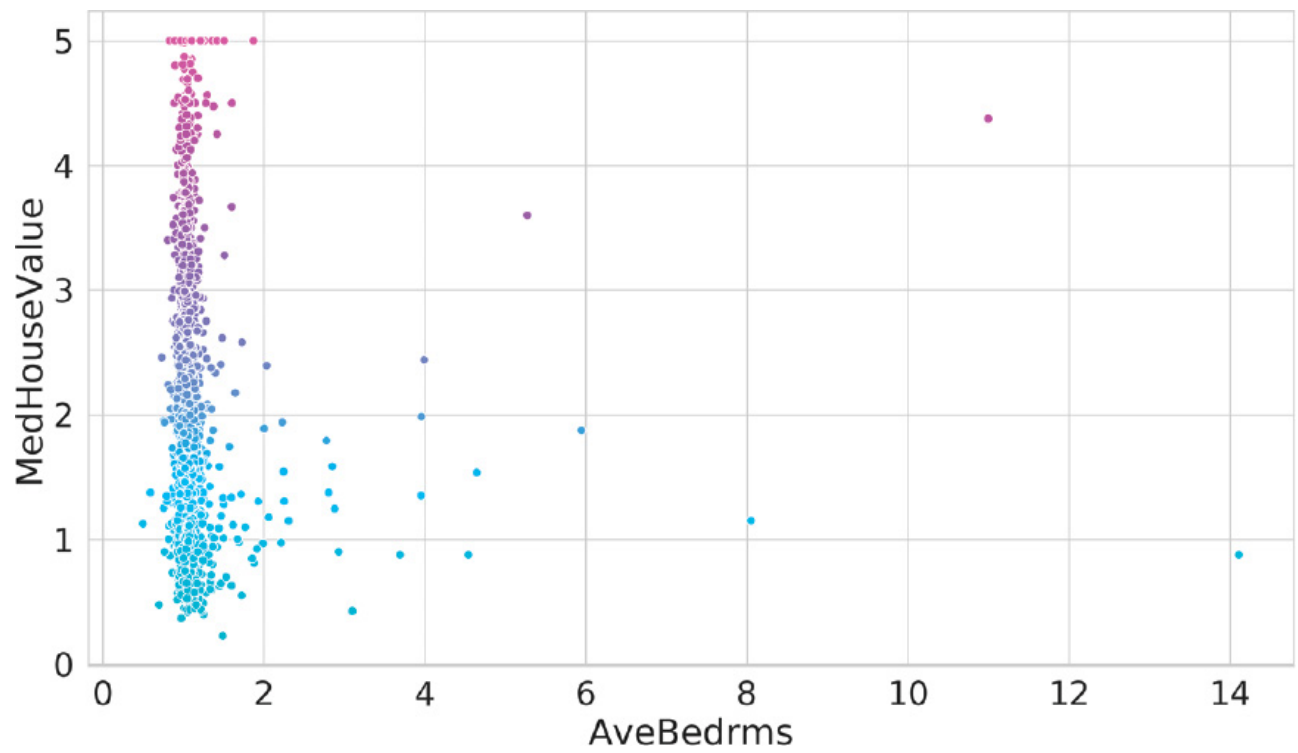
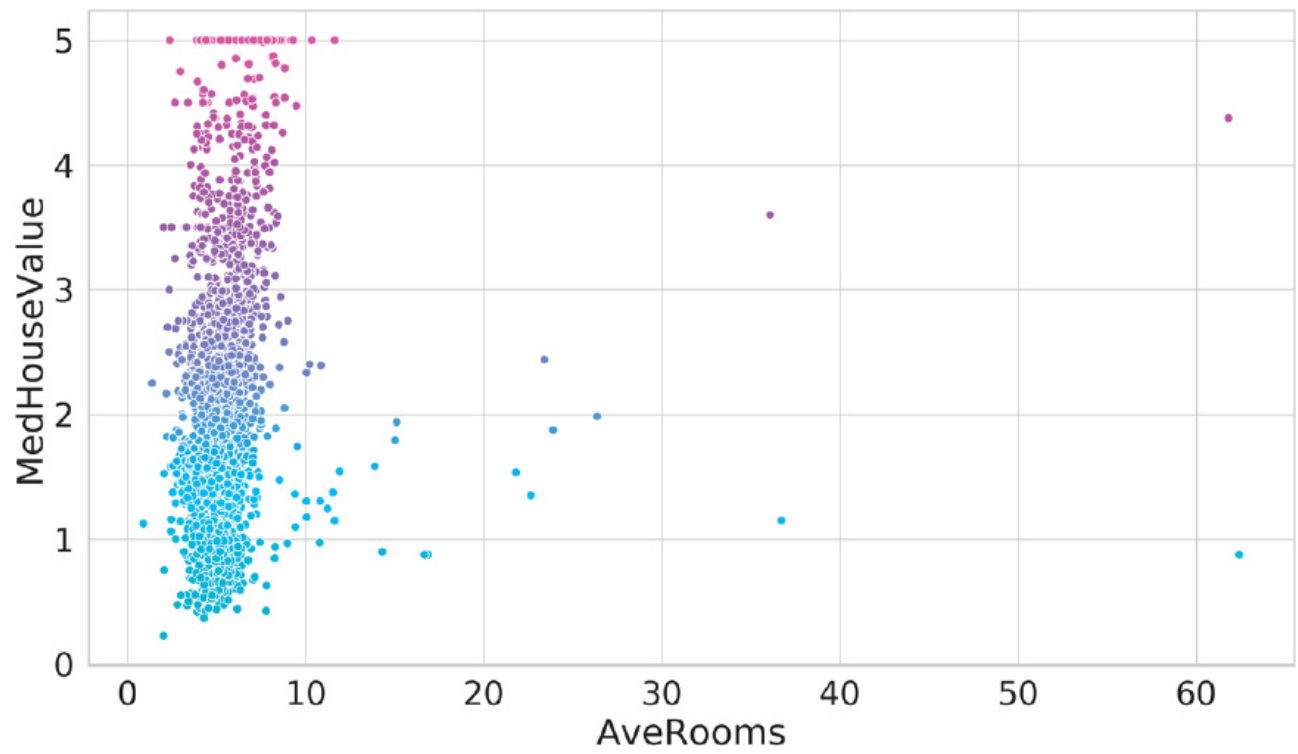
- The graphs showing the latitude and longitude each have two areas of especially significant density. If you search online for the latitude and longitude values where those dense areas appear, you'll see that these represent the greater Los Angeles and greater San Francisco areas where house prices tend to be higher.
- In each graph, there is a horizontal line of dots at the y -axis value 5, which

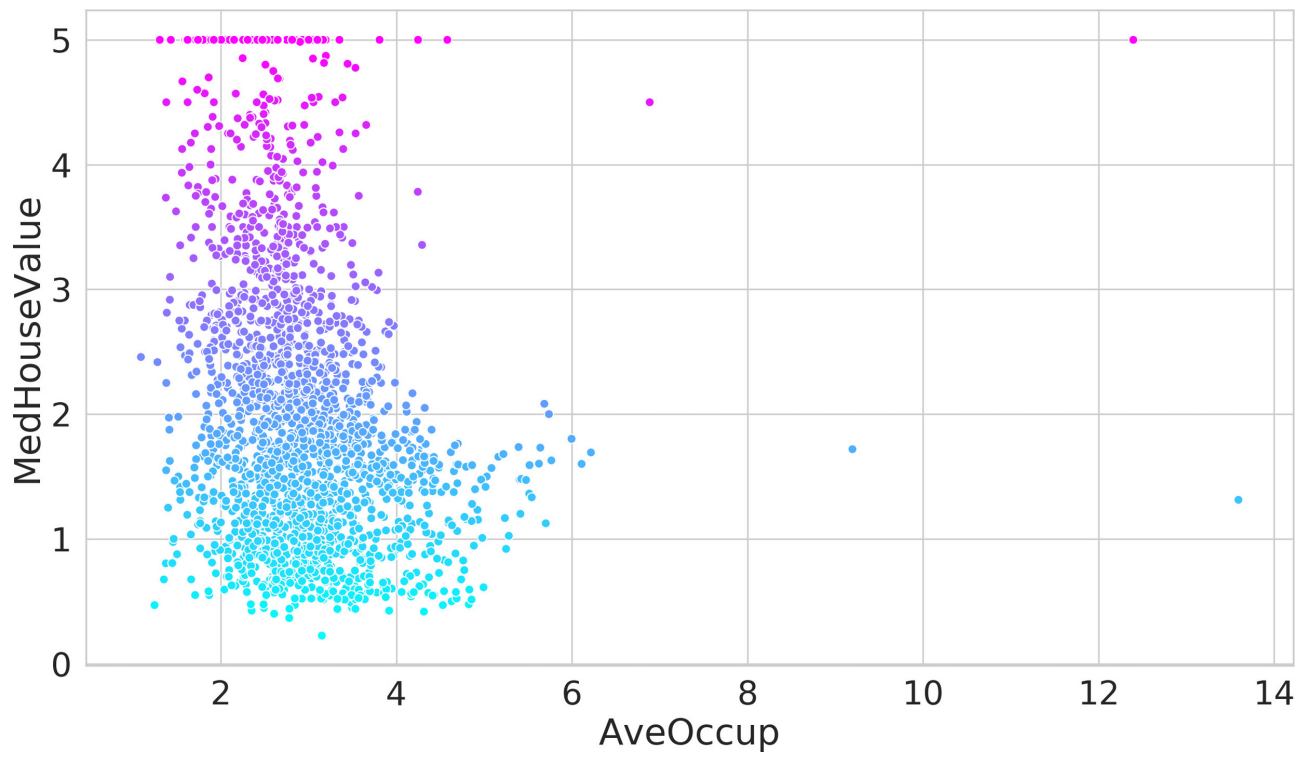
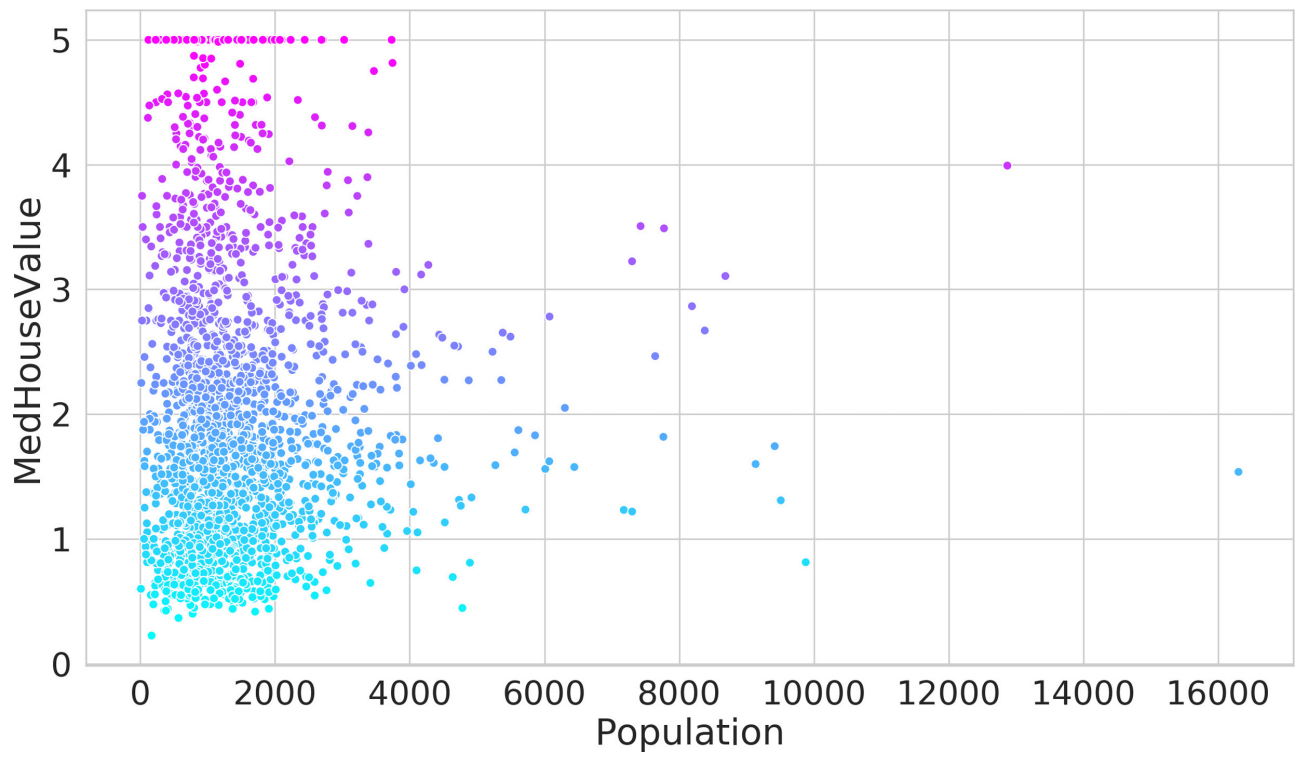
represents the median house value \$500,000. The highest home value that could be chosen on the 1990 census form was “\$500,000 or more.”⁹ So any block group with a median house value over \$500,000 is listed in the dataset as 5. Being able to spot characteristics like this is a compelling reason to do data exploration and visualization.

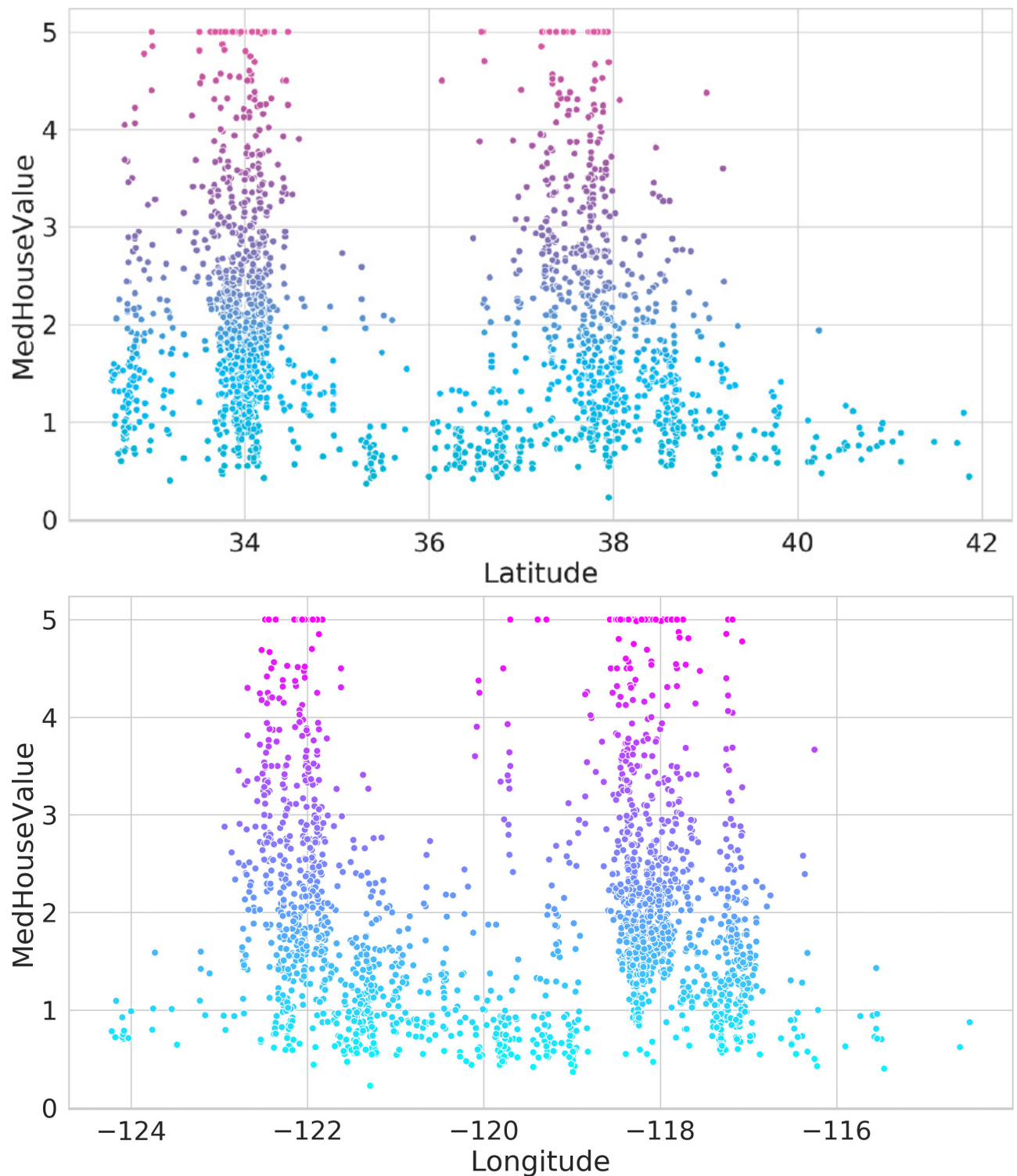
⁹ <https://www.census.gov/prod/1/90dec/cph4/appdx.pdf>.

- In the `HouseAge` graph, there is a vertical line of dots at the `x`-axis value 52. The highest home age that could be chosen on the 1990 census form was 52, so any block group with a median house age over 52 is listed in the dataset as 52.









14.5.4 Splitting the Data for Training and Testing

Once again, to prepare for training and testing the model, let's break the data into training and testing sets using the `train_test_split` function then check their sizes:

[lick here to view code image](#)

```
In [21]: from sklearn.model_selection import train_test_split

In [22]: X_train, X_test, y_train, y_test = train_test_split(
...:     california.data, california.target, random_state=11)
```



```
...:

In [23]: X_train.shape
Out[23]: (15480, 8)

In [24]: X_test.shape
Out[24]: (5160, 8)
```

We used `train_test_split`'s keyword argument `random_state` to seed the random number generator for reproducibility.

14.5.5 Training the Model

Next, we'll train the model. By default, a `LinearRegression` estimator uses *all* the features in the dataset's `data` array to perform a multiple linear regression. An error occurs if any of the features are *categorical* rather than numeric. If a dataset contains categorical data, you either must preprocess the categorical features into numerical ones (which you'll do in the next chapter) or must exclude the categorical features from the training process. A benefit of working with scikit-learn's bundled datasets is that they're already in the correct format for machine learning using scikit-learn's models.

As you saw in the previous two snippets, `X_train` and `X_test` each contain 8 columns—one per feature. Let's create a `LinearRegression` estimator and invoke its `fit` method to train the estimator using `X_train` and `y_train`:

[lick here to view code image](#)

```
In [25]: from sklearn.linear_model import LinearRegression

In [26]: linear_regression = LinearRegression()

In [27]: linear_regression.fit(X=X_train, y=y_train)
Out[27]:
LinearRegression(copy_X=True, fit_intercept=True, n_jobs=None,
                 normalize=False)
```

Multiple linear regression produces separate coefficients for each feature (stored in `coef_`) in the dataset and one intercept (stored in `intercept_`):

[lick here to view code image](#)

```
In [28]: for i, name in enumerate(california.feature_names):
...:     print(f'{name:>10}:    {linear_regression.coef_[i]}')
...:
```

```

MedInc: 0.4377030215382206
HouseAge: 0.009216834565797713
AveRooms: -0.10732526637360985
AveBedrms: 0.611713307391811
Population: -5.756822009298454e-06
AveOccup: -0.0033845664657163703
Latitude: -0.419481860964907
Longitude: -0.4337713349874016

In [29]: linear_regression.intercept_
Out[29]: -36.88295065605547

```

For positive coefficients, the median house value *increases* as the feature value *increases*. For negative coefficients, the median house value *decreases* as the feature value *increases*. Note that the population coefficient has a *negative exponent* ($e-06$), so the coefficient's value is actually -0.000005756822009298454 . This is close to zero, so a block group's population apparently has little effect the median house value.

You can use these values with the following equation to make predictions:

$$y = m_1x_1 + m_2x_2 + \dots m_nx_n + b$$

where

- $m_1, m_2, , m_n$ are the feature coefficients,
- b is the intercept,
- $x_1, x_2, , x_n$ are the feature values (that is, the values of the independent variables), and
- y is the predicted value (that is, the dependent variable).

14.5.6 Testing the Model

Now, let's test the model by calling the estimator's `predict` method with the test samples as an argument. As we've done in each of the previous examples, we store the array of predictions in `predicted` and the array of expected values in `expected`:

[lick here to view code image](#)

```

In [30]: predicted = linear_regression.predict(X_test)

```

```
In [31]: expected = y_test
```

Let's look at the first five predictions and their corresponding expected values:

[lick here to view code image](#)

```
In [32]: predicted[:5]
Out[32]: array([1.25396876, 2.34693107, 2.03794745, 1.8701254 , 2.536083

n [33]: expected[:5]
Out[33]: array([0.762, 1.732, 1.125, 1.37 , 1.856])
```

With classification, we saw that the predictions were distinct classes that matched existing classes in the dataset. With regression, it's tough to get exact predictions, because you have continuous outputs. Every possible value of x_1, x_2, x_n in the calculation

$$y = m_1x_1 + m_2x_2 + \dots m_nx_n + b$$

predicts a value.

14.5.7 Visualizing the Expected vs. Predicted Prices

Let's look at the expected vs. predicted median house values for the test data. First, let's create a DataFrame containing columns for the expected and predicted values:

[lick here to view code image](#)

```
In [34]: df = pd.DataFrame()

In [35]: df['Expected'] = pd.Series(expected)

In [36]: df['Predicted'] = pd.Series(predicted)
```

Now let's plot the data as a scatter plot with the expected (target) prices along the x-axis and the predicted prices along the y-axis:

[lick here to view code image](#)

```
In [37]: figure = plt.figure(figsize=(9, 9))

In [38]: axes = sns.scatterplot(data=df, x='Expected', y='Predicted',
...:                             hue='Predicted', palette='cool', legend=False)
```

...:

Next, let's set the x - and y -axes' limits to use the same scale along both axes:

[lick here to view code image](#)

```
In [39]: start = min(expected.min(), predicted.min())

In [40]: end = max(expected.max(), predicted.max())

In [41]: axes.set_xlim(start, end)
Out[41]: (-0.6830978604144491, 7.155719818496834)

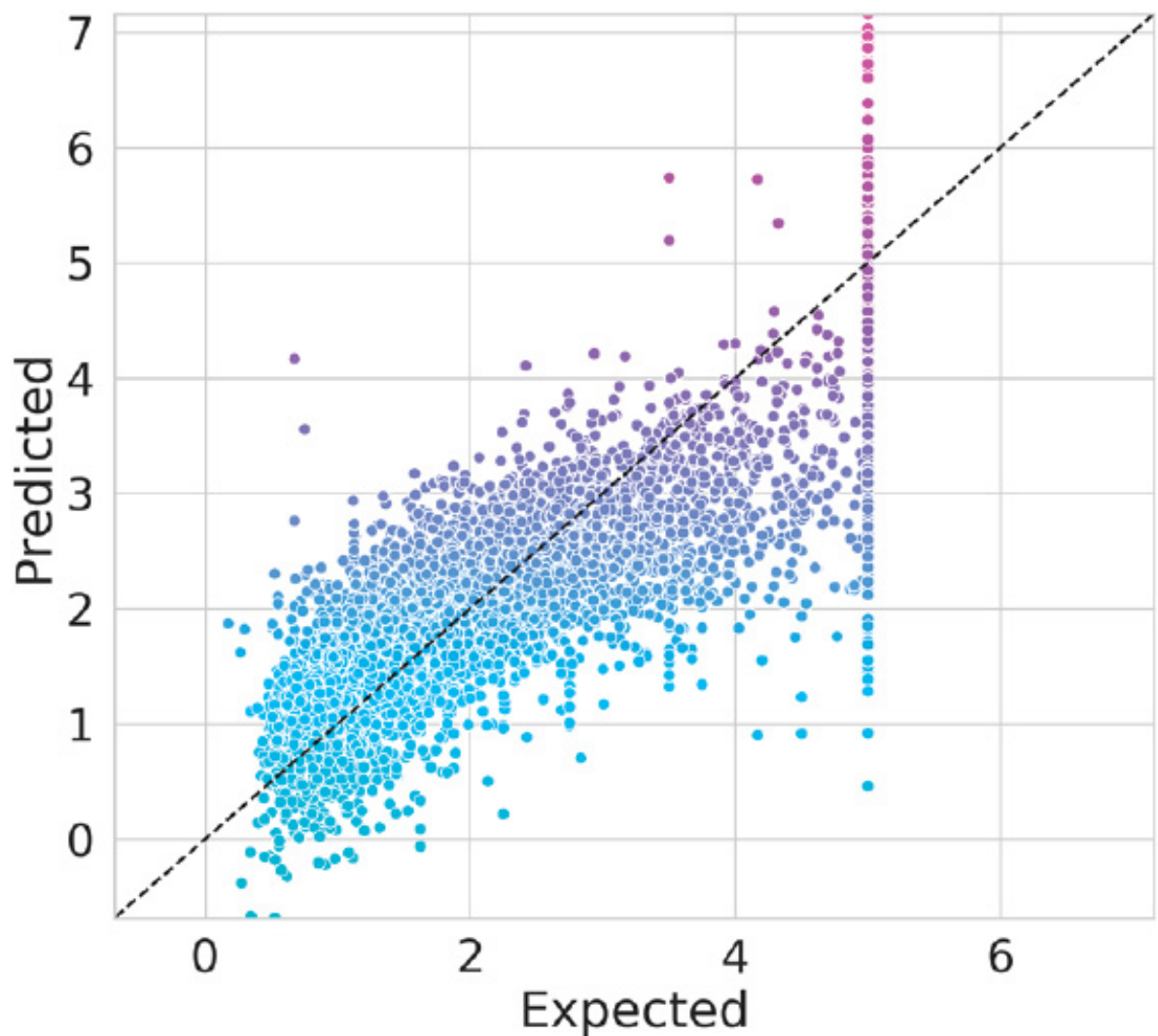
In [42]: axes.set_ylim(start, end)
Out[42]: (-0.6830978604144491, 7.155719818496834)
```

Now, let's plot a line that represents *perfect predictions* (note that this is *not* a regression line). The following snippet displays a line between the points representing the lower-left corner of the graph ($\text{start}, \text{start}$) and the upper-right corner of the graph (end, end). The third argument (`'k--'`) indicates the line's style. The letter `k` represents the color black, and the `--` indicates that plot should draw a dashed line:

[lick here to view code image](#)

```
In [43]: line = plt.plot([start, end], [start, end], 'k--')
```

If every predicted value were to match the expected value, then all the dots would be plotted along the dashed line. In the following diagram, it appears that as the expected median house value increases, more of the predicted values fall below the line. So the model seems to *predict* lower median house values as the *expected* median house value increases.



14.5.8 Regression Model Metrics

Scikit-learn provides many metrics functions for evaluating how well estimators predict results and for comparing estimators to choose the best one(s) for your particular study. These metrics vary by estimator type. For example, the `sklearn.metrics` functions `confusion_matrix` and `classification_report` used in the Digits dataset classification case study are two of many metrics functions specifically for evaluating *classification* estimators.

Among the many metrics for regression estimators is the model's **coefficient of determination**, which is also called the **R^2 score**. To calculate an estimator's R^2 score, call the `sklearn.metrics` module's **`r2_score`** function with the arrays representing the expected and predicted results:

[lick here to view code image](#)

```
In [44]: from sklearn import metrics

In [45]: metrics.r2_score(expected, predicted)
Out[45]: 0.6008983115964333
```

R^2 scores range from 0.0 to 1.0 with 1.0 being the best. An R^2 score of 1.0 indicates that the estimator perfectly predicts the dependent variable's value, given the independent variable(s) value(s). An R^2 score of 0.0 indicates the model cannot make predictions with any accuracy, based on the independent variables' values.

Another common metric for regression models is the **mean squared error**, which

- calculates the difference between each expected and predicted value—this is called the *error*,
- squares each difference and
- calculates the average of the squared values.

To calculate an estimator's mean squared error, call function **mean_squared_error** (from module `sklearn.metrics`) with the arrays representing the expected and predicted results:

[lick here to view code image](#)

```
In [46]: metrics.mean_squared_error(expected, predicted)
Out[46]: 0.5350149774449119
```

When comparing estimators with the mean squared error metric, the one with the value closest to 0 best fits your data. In the next section, we'll run several regression estimators using the California Housing dataset. For the list of scikit-learn's metrics functions by estimator category, see

https://scikit-learn.org/stable/modules/model_evaluation.html

14.5.9 Choosing the Best Model

As we did in the classification case study, let's try several estimators to determine whether any produces better results than the `LinearRegression` estimator. In this example, we'll use the `linear_regression` estimator we already created as well as `ElasticNet`, `Lasso` and `Ridge` regression estimators (all from the `sklearn.linear_model` module). For information about these estimators, see

https://scikit-learn.org/stable/modules/linear_model.html

[lick here to view code image](#)

```
In [47]: from sklearn.linear_model import ElasticNet, Lasso, Ridge

In [48]: estimators = {
...:     'LinearRegression': linear_regression,
...:     'ElasticNet': ElasticNet(),
...:     'Lasso': Lasso(),
...:     'Ridge': Ridge()
...: }
```

Once again, we'll run the estimators using k-fold cross-validation with a `KFold` object and the `cross_val_score` function. Here, we pass to `cross_val_score` the additional keyword argument `scoring='r2'`, which indicates that the function should report the R^2 scores for each fold—again, 1.0 is the best, so it appears that `LinearRegression` and `Ridge` are the best models for this dataset:

[lick here to view code image](#)

```
In [49]: from sklearn.model_selection import KFold, cross_val_score

In [50]: for estimator_name, estimator_object in estimators.items():
...:     kfold = KFold(n_splits=10, random_state=11, shuffle=True)
...:     scores = cross_val_score(estimator=estimator_object,
...:                               X=california.data, y=california.target, cv=kfold,
...:                               scoring='r2')
...:     print(f'{estimator_name:>16}: ' +
...:           f'mean of r2 scores={scores.mean():.3f}')
...:
LinearRegression: mean of r2 scores=0.599
ElasticNet: mean of r2 scores=0.423
Lasso: mean of r2 scores=0.285
Ridge: mean of r2 scores=0.599
```

14.6 CASE STUDY: UNSUPERVISED MACHINE LEARNING, PART 1—DIMENSIONALITY REDUCTION

In our data science presentations, we've focused on getting to know your data.

Unsupervised machine learning and visualization can help you do this by finding patterns and relationships among unlabeled samples.

For datasets like the univariate time series we used earlier in this chapter, visualizing the data is easy. In that case, we had two variables—date and temperature—so we

plotted the data in two dimensions with one variable along each axis. Using Matplotlib, Seaborn and other visualization libraries, you also can plot datasets with three variables using 3D visualizations. But how do you visualize data with more than three dimensions? For example, in the Digits dataset, every sample has 64 features and a target value. In big data, samples can have hundreds, thousands or even millions of features.

To visualize a dataset with many features (that is, many dimensions), we'll first *reduce* the data to two or three dimensions. This requires an unsupervised machine learning technique called **dimensionality reduction**. When you graph the resulting information, you might see patterns in the data that will help you choose the most appropriate machine learning algorithms to use. For example, if the visualization contains *clusters* of points, it might indicate that there are distinct classes of information within the dataset. So a classification algorithm might be appropriate. Of course, you'd first need to determine the class of the samples in each cluster. This might require studying the samples in a cluster to see what they have in common.

Dimensionality reduction also serves other purposes. Training estimators on big data with significant numbers of dimensions can take hours, days, weeks or longer. It's also difficult for humans to think about data with large numbers of dimensions. This is called the **curse of dimensionality**. If the data has closely correlated features, some could be eliminated via dimensionality reduction to improve the training performance. This, however, might reduce the accuracy of the model.

Recall that the Digits dataset is already labeled with 10 classes representing the digits 0–9. Let's ignore those labels and use dimensionality reduction to reduce the dataset's features to two dimensions, so we can visualize the resulting data.

Loading the Digits Dataset

Launch IPython with:

```
ipython --matplotlib
```

then load the dataset:

[lick here to view code image](#)

```
In [1]: from sklearn.datasets import load_digits

In [2]: digits = load_digits()
```

Creating a TSNE Estimator for Dimensionality Reduction

Next, we'll use the **TSNE estimator** (from the `sklearn.manifold` module) to perform dimensionality reduction. This estimator uses an algorithm called t-distributed Stochastic Neighbor Embedding (t-SNE) ⁰ to analyze a dataset's features and reduce them to the specified number of dimensions. We first tried the popular PCA (principal components analysis) estimator but did not like the results we were getting, so we switched to TSNE. We'll show PCA later in this case study.

⁰The algorithms details are beyond this books scope. For more information, see <https://scikit-learn.org/stable/modules/manifold.html#t-sne>.

Let's create a TSNE object for reducing a dataset's features to two dimensions, as specified by the keyword argument `n_components`. As with the other estimators we've presented, we used the `random_state` keyword argument to ensure the reproducibility of the "render sequence" when we display the digit clusters:

[lick here to view code image](#)

```
In [3]: from sklearn.manifold import TSNE

In [4]: tsne = TSNE(n_components=2,    random_state=11)
```

Transforming the Digits Dataset's Features into Two Dimensions

Dimensionality reduction in scikit-learn typically involves two steps—training the estimator with the dataset, then using the estimator to transform the data into the specified number of dimensions. These steps can be performed separately with the TSNE methods `fit` and `transform`, or they can be performed in one statement using the `fit_transform` method: ¹

¹Every call to `fit_transform` trains the estimator. If you intend to reuse the estimator to reduce the dimensions of samples multiple times, use `fit` to once train the estimator, then use `transform` to perform the reductions. Well use this technique with PCA later in this case study.

[lick here to view code image](#)

```
In [5]: reduced_data = tsne.fit_transform(digits.data)
```

TSNE's `fit_transform` method takes some time to train the estimator then perform the reduction. On our system, this took about 20 seconds. When the method completes its task, it returns an array with the same number of rows as `digits.data`, but only two columns. You can confirm this by checking `reduced_data`'s shape:

[lick here to view code image](#)

```
In [6]: reduced_data.shape
Out[6]: (1797, 2)
```

Visualizing the Reduced Data

Now that we've reduced the original dataset to only two dimensions, let's use a scatter plot to display the data. In this case, rather than Seaborn's `scatterplot` function, we'll use Matplotlib's **`scatter` function**, because it returns a collection of the plotted items. We'll use that feature in a second scatter plot momentarily:

[lick here to view code image](#)

```
In [7]: import matplotlib.pyplot as plt

In [8]: dots = plt.scatter(reduced_data[:, 0], reduced_data[:, 1],
...:                       c='black')
...:
```

Function `scatter`'s first two arguments are `reduced_data`'s columns (0 and 1) containing the data for the x - and y -axes. The keyword argument `c='black'` specifies the color of the dots. We did not label the axes, because they do not correspond to specific features of the original dataset. The new features produced by the TSNE estimator could be quite different from the dataset's original features.

The following diagram shows the resulting scatter plot. There are clearly *clusters* of related data points, though there appear to be 11 main clusters, rather than 10. There also are “loose” data points that do not appear to be part of specific clusters. Based on our earlier- study of the Digits dataset this makes sense because some digits were difficult to classify.